CONEXANT™

iTVC15™ MPEG Codec Firmware API Reference Manual

Revision 1.2

Confidential NDA Required

Conexant Systems, Inc.
Corporate Headquarters
4311 Jamboree Road
Newport Beach, California, 92658-8902
tel: 949.483.4600
fax: 949.483.6375
web: www.conexant.com

Silicon Valley Office
2500 Walsh Avenue
Santa Clara, California, 95051
tel: 408.727.5077
fax: 408.727.5078

Pub Date: July 2002
Reference Number: DC032-FW004

This document provides information on the API used in conjunction with Conexant's iTVC15 MPEG codec and will remain the official reference source for all revisions/releases of this product until rescinded by an update.

Revision History

Revision	Date	Notes
1.2	July 18, 2002	Formatting has been modified to reflect the acquisition of GlobespanVirata's Video Compression Group by Conexant Systems, Inc.
1.1	June 28, 2002	Typical periodic update.
1.0	February 21, 2002	APIs updated. Reorganization. Name change of manual. See Revision History section for details.
0.996	August 2, 2001	Typical periodic update.
0.995	April 30, 2001	Typical periodic update.
0.994	January 31, 2001	Initial customer-ready release.

REVISION HISTORY

Changes Since Previous Revision (1.1)

1. Formatting has been modified to reflect the acquisition of GlobespanVirata's Video Compression Group by Conexant Systems, Inc.

TABLE OF CONTENTS

List of Figures

LIST OF TABLES

SECTION 1
ABOUT THIS DOCUMENT

1.1 Overview

This document describes the firmware application programming interface (API) for Conexant's iTVC15 MPEG codec. It includes information on the software architecture of the iTVC15, and provides descriptions of the event notifications and API functions used in the encoder, decoder, and display blocks of the chip. Documentation is available for the middleware and application layer demonstration software used in the various iTVC15 reference designs (see Section 1.3 for details).

1.2 Scope and Structure

The scope of this document extends only to the API used in conjunction with Conexant's iTVC15 MPEG codec. The document is organized into the following chapters:

About This Document. This section provides information that may help you understand the material presented in this document. This includes listings of related documentation, conventions used in the document, a list of acronyms, and contact information.

Introduction. This section provides some very general information on the structure of the software and firmware API. Also included is information on how the firmware is downloaded and how commands are sent to the firmware, including the structure of the mailbox, MCI address mapping, etc. The handling of event notifications and data transfers is also covered.

Operational Units. This section describes the two of the basic functions of the firmware, encoding and decoding. For each of these functions, programming the iTVC15, programming the audio/video peripherals, and controlling the session are covered.

Event Notifications. This section documents the individual event notifications received from the firmware.

API Commands. This section documents all of the API commands used by the firmware, which are grouped by function: encoder, decoder, and display.

1.3 Conexant Technical Publications

The documents in Table 1-1 below are intended for use in conjunction with the *iTVC15 Firmware API Reference Manual*. This documentation is distributed as part of a Design Package, which in addition to hardware includes release firmware, schematics, sample code, layout gerbers, Bill of Materials, and other related material. This specification is intended as a reference source only, not as a step-by-step guide for building a product. **Additional documentation may be available** if you have purchased one of the Conexant reference designs built for the iTVC15 MPEG codec.

Table 1-1: Related Documentation

Document Name	Reference Number
iTVC15 Data Sheet	DC031-SYS007
iTVC15 Application Notes	Each App Note referenced individually
App Note 006: Interfacing with iTVC15 Drivers and DirectShow Filters	DC055-APP027
iTVC15 Host Driver Development Reference	DC102-SW007
iTVC15 Time-shift Library Specification for Linux	DC058-SW003
iTVC15 PCI Device Driver Development Reference for Linux	DC059-SW004

1.4 Conventions

Active-LOW Signals. Signal names containing the # sign (e.g., REQ#) indicate active-LOW signals. They are asserted in their low-voltage state and negated in their high-voltage state. When used in this context, HIGH and LOW are written in upper case letters.

Addresses. All addresses are expressed in hexadecimal and point to bytes.

Binary Numbers. Binary numbers are always indicated in their standard format, except in special instances when they will be denoted by the suffix 'b' for clarity. See "Hexadecimal Numbers" for information on how hexadecimal numbers are handled.

Bit Ranges. In text, bit ranges are shown with an en-dash (e.g., bits 31–0). When accompanied by a signal or bus name, the highest and lowest bit numbers are contained in brackets and separated by a colon (e.g., SSDATA[31:0]). This document employes little-endian bit ordering, meaning the most significant bit is listed first and is always the largest number of any bit range.

Bit Values. Single bits can be either set to '1' or cleared to '0'. Multiple bits are expressed in double quotes (e.g., "1001").

Hexadecimal Numbers. Hexadecimal numbers are indicated by the prefix '0x' (e.g., 0x0048).

Object Size Notation. The following are the conventions used to denote object size:

- *byte* — 8-bit object
- *word* — 16-bit (2-byte) object
- *dword* — 32-bit (4-byte) object

Signal Ranges. In a range of signals, the highest and lowest signal numbers are contained in brackets and separated by a colon (e.g., SSDATA[31:0]). This document employes little-endian bit ordering, meaning the most significant bit is listed first and is always the largest number of any signal range.

Tri-State. In timing diagrams, signal ranges that are high-impedance are shown as a straight horizontal line between the high and low levels.

1.5 List of Acronyms

API	application programming interface
APU	audio processing unit
ARGB	alpha red-green-blue
BLT	block transfer
CCIR	Comite' Consultatif International des Radio Communications
CCITT	Comite' Consultatif International Telegraphique et Telephonique
CLUT	color lookup table
CPU	central processing unit
CRC	cyclic redundancy check
DMA	direct memory access
DMAC	DMA controller
DNR	dynamic noise reduction
DVD	digital versatile disc
ES	elementary stream
FIFO	first in / first out
FRM	Flicker Reduction Module
GOP	group of pictures
GUI	graphical user interface
I^2C	Inter-Integrated Circuit
ISR	interrupt service routine
M1S	MPEG-1 system
MCI	Microcontroller Interface
MPEG	Moving Pictures Experts Group
MSB	most significant bit
NTSC	National Television Standards Committee
OSD	on-screen display
PAL	Phase Alternation Line
PCI	Peripheral Component Interconnect
PCM	pulse code modulation
PES	packetized elementary stream
PS	Program stream
PTS	presentation time stamp
RGB	red-green-blue
ROP	raster operation
SPU	system processing unit
TS	Transport stream
TV	television
VBI	vertical blanking interval
VIM	Video Input Module
VPU	video processing unit

1.6 Contact Information

For additional information, contact Conexant Systems, Inc. at 1-408-727-5077, or visit the Conexant Web site located at http://www.conexant.com.

Access to technical support is available if you have already purchased a reference design from Conexant.

Updates to the iTVC15 document set are available on Conexant's Video Compression Group "ftp" site. Ask your representative for details.

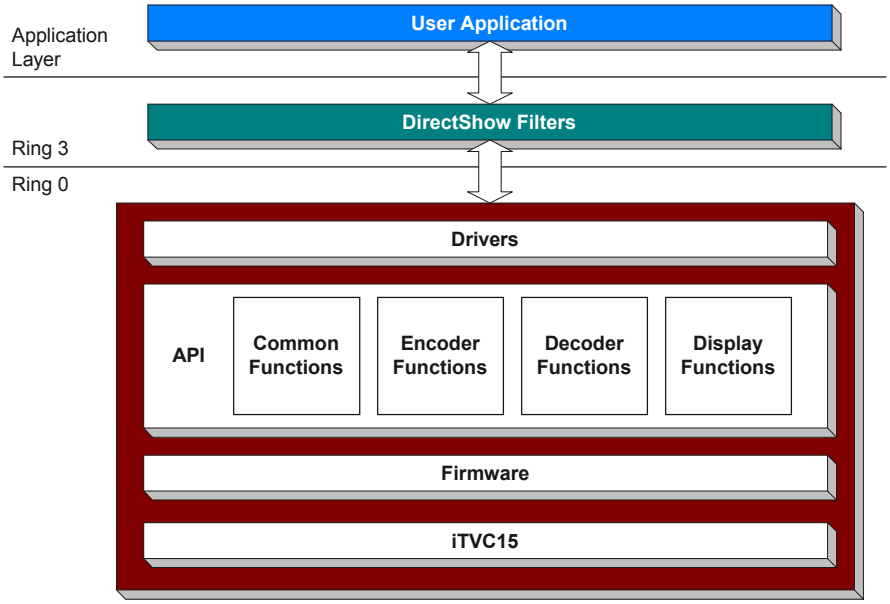
SECTION 2
INTRODUCTION

2.1 Introduction

2.1.1 Software Architecture

A complete iTVC15 solution will typically consist of three software layers. The lowest level is the firmware (developed by Conexant) that resides within the iTVC15 chip and associated memory. The next layer is the driver (developed by Conexant) layer, which interfaces with the firmware through an API that is described in this document. The highest layer is the user application, developed by the customer, which interfaces with the iTVC15 through the driver. Depending on the overall system configuration, the graphical user interface (GUI) can be provided to the end-user through the iTVC15 on-screen display (OSD), again via the API.

Figure 2-1: iTVC15 Software Architecture



2.1.2 Downloading the Firmware

The iTVC15 relies on firmware for its functionality. This firmware is provided by Conexant, and different versions of firmware may be developed to provide different functionality.

Before any encoding or decoding can happen, the firmware must be downloaded to the iTVC15. The host loads the firmware over PCI or the microcontroller port.

2.1.3 Configuring the Environment

Before any audio or video data can be processed, the customer-specific environment must be configured. This includes all of the audio/video peripherals that are connected to the iTVC15. Typically this involves *I^2C programming*. This type of programming is done by the customer. The iTVC15 hardware provides an on-chip I^2C controller and an associated API which can be used for this purpose.

2.1.4 Firmware API

Once the environment is configured and the iTVC15 firmware initialized, an encoding and/or decoding session can begin. The firmware provides an API for setting parameters and controlling the session via start, stop, etc. In addition, a very capable OSD that is integral to the decoder and supported through the API allows a GUI to be presented on the same screen as the decoded video.

2.1.5 Data Transfers

The encoder compresses audio and video into an MPEG system-level stream that can be sent to the MPEG streaming port or Peripheral Component Interconnect (PCI). The system-level streams supported include packetized elementary stream (PES), Program stream (PS), Transport stream (TS), and MPEG-1 system stream (M1S).

The decoder similarly consumes a system-level stream that is provided via the MPEG streaming data port or PCI. In either case, the actual port used for data transfers is selected via an API call.

2.1.6 Interrupts

Interrupts are generated to inform the driver of certain events related to video timing, OSD block transfer (BLT) tasks, encoded data buffer availability, etc.

2.2 Downloading the Firmware

The firmware needs to be downloaded when the chip is reset. The host can reset the chip and load the firmware over the PCI bus or via the microcontroller port. Both the PCI and the microcontroller interface have full access to the entire memory and register space, and therefore the firmware downloading mechanism is more or less identical for these platforms.

The iTVC15 is configured in PCI mode (PI_SEL signal of iTVC15 is high) or non-PCI mode (PI_SEL is low). In PCI mode, the firmware is loaded over the PCI bus. In non-PCI mode, the firmware is loaded through the microcontroller interface.

The firmware contains code for three different Java processors in the iTVC15 — the video processing unit (VPU), audio processing unit (APU), and system processing unit (SPU). The APU and SPU share the same memory space, and the VPU is connected to a different memory space. By design, the SPU gets out of reset first, and then it can enable the VPU and the APU through the chip reset register. Instead of separately downloading the firmware in the

VPU code space and the APU/SPU code space, it can be downloaded only in SPU code space, and then the SPU can use DMA transfer to download the data to the VPU code space before enabling the VPU.

2.3 Sending Commands to Firmware

2.3.1 Introduction

An MPEG-2 system requires on-going interaction between an upper-level player/recorder application and the low-level firmware. The high-level application asks the firmware to start/pause/stop the encoding and/or decoding. Before starting encoding, a number of encoding parameters like video bit rate, video output resolution, audio sampling rate, audio bit rate, etc. have to be programmed. Even the decoder has to be configured for input source (PCI memory or streaming port).

2.3.2 APIs Instead of Direct Register Access

It is possible to set most of these parameters by directly writing to iTVC15 registers. However, such an implementation runs the risk of failing if in a subsequent hardware revision the register addresses or the relevant bits get changed and the application does not get updated. A safer system design is to allow the application and firmware to interact through a set of API function calls. The API functions provide a level of abstraction for the hardware details.

The firmware mailbox is a very simple encapsulation of an API request. The firmware mailboxes reside inside the iTVC15's memory.

The mailbox commands can be sent via PCI or the microcontroller interface (MCI). PCI provides the most capable command set and is a superset of the commands available using the MCI. However, when a command is supported via the MCI, the command, its parameters, and the messaging mechanisms are identical to those for PCI.

2.3.3 Firmware Mailbox

A mailbox is nothing but 80 bytes (20 dwords) of contiguous memory space, defined as follows:

Table 2-1: Firmware Mailbox Structure

Address	Data Bits 31–0
0	Mailbox flags
4	Command code
8	Return code
12	Timeout
16	Parameter 1 / Result 1
20	Parameter 2 / Result 2
24	Parameter 3 / Result 3
28	Parameter 4 / Result 4
32	Parameter 5 / Result 5
36	Parameter 6 / Result 6
40	Parameter 7 / Result 7
44	Parameter 8 / Result 8
48	Parameter 9 / Result 9
52	Parameter 10 / Result 10
56	Parameter 11 / Result 11
60	Parameter 12 / Result 12
64	Parameter 13 / Result 13
68	Parameter 14 / Result 14
72	Parameter 15 / Result 15
76	Parameter 16 / Result 16

The current firmware implementation supports 10 such mailboxes (or slots) inside iTVC15's memory.

Every API function has a unique command code. In order to issue a particular API function request, the driver writes the command code in location 4, and all the associated parameters in locations 16–76. The firmware interprets the command code, takes the appropriate action, then returns the results in the same locations 16–76 (i.e., the results overwrite the parameters).

2.3.3.1 Mailbox Flag

The mailbox flag acts as a state machine for the mailbox API procedure. Only the three least significant bits of this flag are used:

- Bit 0 specifies whether the slot is empty (0) or full (1).
- Bit 1 specifies whether the parameters are ready (1) or not (0).
- Bit 2 specifies whether the firmware has processed the slot (1) or not (0).

The mailbox flag is set by the driver to specify that a command is ready to be processed (mailbox flag bits 0 and 1 are 1). After processing the call, the firmware changes the value to specify that the command has been processed (mailbox flag bits 0–2 are all 1s).

2.3.3.2 Command Code

The four-byte command code is specified at location 4. An enumerator is defined in the file doapi.h to specify each command's unique code.

2.3.3.3 Input Parameters (P1 – P16)

Each mailbox command can optionally take up to 16 input parameters (P1–P16) at locations 16–76. Each parameter location is four bytes wide. The host application uses the parameter locations to specify the API parameters.

2.3.3.4 Return Code

Four bytes are reserved at location 8 for the return code. Currently, the firmware sets the return code to one of two values: API_RC_SUCCESS (0) if the command succeeds or API_RC_ERROR_UNKNOWN (–1) if the command fails.

2.3.3.5 Return Values (R1 – R16)

Each mailbox command can optionally return up to 16 result values (R1–R16) at locations 16–76. Each return location is four bytes wide. The firmware uses the same locations as the input parameters to return the results. The number of result values, if any, is specific to the API command and does not usually match the number of input parameters.

2.3.3.6 Timeout

There is also a four-byte timeout field at location 12. This is a timeout to prevent a mailbox from remaining occupied indefinitely if the host application crashes or gets stuck in an infinite loop. The application sets its timeout before sending the command. While checking for an empty mailbox, the firmware also checks for all the mailboxes which have been processed by the firmware but not released by the host application.

The complete procedure for sending an API command request to the firmware from the driver's perspective is as follows {function ExecuteApiRequest() of file doapi.c}:

- Loop through the 10 command slots to find an empty slot (bit 0 of the flag is zero).
- Change the flag of this slot to full (bit 0 of the flag is set to 1).
- Write the command bytes into that slot and set the flag to ready (bits 0–1 of the flag are set to 1). It is not required to write the entire 80 bytes, only the parameters relevant to the particular API have to be set.
- Poll the flag until it specifies that the firmware has finished processing the command (bits 0–2 of the flag are set to 1).
- Read the mailbox bytes back to get the return code and the result bytes.

Note: This procedure is subject to change for iTVC15 to handle potential race conditions between different threads accessing the mailboxes for different sub-functions within an application (e.g., encoding, decoding, display).

2.3.3.7 Accessing the Mailboxes

PCI

The simplest implementation of the mailbox mechanism is for the PCI environment. The iTVC15's memory is directly accessible to the driver through the PCI interface. The initial part of the memory is designated as the firmware identification block (refer to file 'fwid.h' for a definition), which contains several useful pieces of information including firmware features, version number, and release date. The host reads this firmware information block to find out the absolute address of the mailboxes. Once the absolute address is known, the driver can directly manipulate the mailboxes to set up an API call.

Microcontroller

Please refer to iTVC15 Application Note 014, *iTVC15 Microcontroller Interface,* for more information on how the iTVC15 operates when in microcontroller mode.

Other than the low-level protocol for reading from / writing to memory, the overall mailbox mechanism remains the same under the microcontroller environment. The microcontroller driver first reads the firmware information block to determine the mailbox address. Then it manipulates the mailboxes directly to set up API calls.

Under the iTVC15, the MCI has full access to the entire memory space as well as the register space using only an 8-bit data bus and a 5-bit address bus. A complete memory or register read/write normally requires 7 MCI cycles.

To write data to memory, first MCI locations 0–3 are written with 32 bits of data. Then MCI locations 4–6 are written with 22 bits of address. Bit 22 of location 4 has to be set to 1 for a write cycle. When location 6 is written, the output ready signal is asserted high and the actual memory write cycle begins.

To read data from memory, MCI locations 4–6 are written with 22 bits of address, setting bit 22 of location 4 to 0 for a read cycle. When location 6 is written, the output ready signal is asserted high and the actual memory read cycle begins. The microcontroller can read the data back from locations 0–3 after the ready signal returns low.

The following table summarizes how each MCI address is used to transfer address/data/control information to the iTVC15.

Table 2-2: MCI Address Mapping

MCI Address [4:0]	MCI Data [7:0]
0	Memory Data [7:0]
1	Memory Data [15:8]
2	Memory Data [23:16]
3	Memory Data [31:24]
4	[7] = Must set to '0' [6] = mode: Read = 0 Write = 1 [5:0] = Memory Address [21:16]
5	Memory Address [15:8]
6	Memory Address [7:0]
7	[7] = Reserved [6:4] = 0b000 [3] = ready [2] = memory select: SDF = 0 SDS = 1 [1] = use interrupt pin 1 [0] = use interrupt pin 2
8	Register Data [7:0]
9	Register Data [15:8]
10 (0x0A)	Register Data [23:16]
11 (0x0B)	Register Data [31:24]
12 (0x0C)	Register Address [7:0]
13 (0x0D)	Register Address [15:8]
14 (0x0E)	[0] = mode Read = 0 Write = 1
31–15 (0x0F)	Not used

2.3.4 Order of Execution

The iTVC15 has three distinct operational units within the device:

1. MPEG Encoder
2. MPEG Decoder
3. Display

Internally, three queues of pending commands are maintained corresponding to these three functional units. API commands submitted to an individual unit will be executed strictly in order. API commands submitted to two independent units may execute out of order relative to each other. When it is necessary to synchronize activities across functional units, specific APIs are created. The API commands are listed in the appendices with one for each functional unit.

2.4 Event Notification from Firmware

2.4.1 Introduction

In addition to API commands, the iTVC15 mailbox architecture provides a mechanism for setting up notifications for specific events, which signal the occurrence of the event and communicate information to the host about the event. The occurrence of the event is signaled through an interrupt and information about the event may quickly be obtained from a dedicated mailbox slot.

The mailbox location and the type of the event are selected by the host through the API command SetupEventNotification. Event notifications are generated continuously until cancelled by the host through the API command CancelEventNotification (or other API commands which may cancel the operation for which the notification is requested — i.e., StopCapture stops all event notifications for the encoder).

Mailbox slots for event notifications are separate from API mailboxes and are allocated in a contiguous space immediately following the API mailboxes. The structure of the mailbox slot is the same as that used for API commands, except that the first four dwords are ignored and only the Result dwords are read back as return parameters. Currently, 10 mailbox slots are allocated for event notifications, which means that up to 10 simultaneous event notifications may be set up by the host. It is assumed that once the interrupt is received by the host, all of the return parameters have been correctly filled in the mailbox by the firmware, and that when the host clears the interrupt, all of the parameters have been successfully received by the host. The return parameters for each event are defined later in this document.

2.4.2 Basic Steps for Handling Event Notifications

Step 1: Query the Firmware for Event Notification Results

- The host reads the interrupt status register (0x40) to determine the event for which the interrupt was generated. Many bits may be asserted by the firmware if interrupts are pending for various event notifications. The host may decide either to handle the interrupts sequentially (by servicing and clearing them one at a time), or handle them at the same time (by servicing all and clearing them during the same ISR).
- The host reads the return parameters of the event notification.

Step 2: Clear the Interrupt Status Register Bits

- Clear the bits in the interrupt status register for which the interrupt was handled.

2.4.3 Event Notification APIs

2.4.3.1 Setup

This API provides the firmware with information about the type of event notification that is desired and the mailbox location where the returned information should be placed, as well as the location of the bit representing the event in the interrupt status register.

`SetEventNotification`

2.4.3.2 Cancel

This API cancels any pending event notifications for the specified event, and no future interrupts are sent to the host for that event.

`CancelEventNotification`

2.5 Data Transfers

2.5.1 PCI Data Transfers

PCI data transfers are all direct memory access (DMA) transfers. The following terminology will be used throughout this section:

- ***ITVC15 Memory.*** This is the total amount of SDRAM available to the iTVC15, all of which is mapped to the PCI addressing space.
- ***Host Memory.*** This is the total amount of memory available to the host CPU which is accessible to PCI.
- ***DMA to Host.*** This is a DMA transfer from ITVC15 memory to host memory.
- ***DMA from Host.*** This is a DMA transfer from host memory to iTVC15 memory.

2.5.2 General Procedures for DMA

The host creates a DMA-linked list in host memory which is read by the firmware to control a DMA transfer that can be to or from host memory. All DMA transfers must be aligned on a 4-byte boundary and each DMA can be constructed as a single link or multi-link transfer.

Each link list entry in the DMA link list is a 12-byte structure containing the Source Address, Destination Address, Number of Bytes to Transfer, and Last Flag. The last link in the list must have the Last Flag bit set to 1 to ensure that the DMA controller (DMAC) will generate an interrupt to the host when the transfer for this link completes.

<table>
<tr><th>31</th><th>30</th><th></th><th>18</th><th>17</th><th></th><th>0</th></tr>
<tr><td colspan="7">Source Address</td></tr>
<tr><td colspan="7">Destination Address</td></tr>
<tr><td>Last</td><td colspan="3">Reserved</td><td colspan="3">Byte count</td></tr>
</table>

The host address is known to the host and is used as the Source (Destination) address depending on the direction of the DMA transfer. The second address is an address in the iTVC15, and its value (offset within SDS or SDF memory) is communicated to the host via an event notification after completing the previous transfer. The physical address is computed by adding the appropriate PCI base to this iTVC15 offset, and this is then used as the Destination (Source) address in the link list entry.

2.5.2.1 Basic Steps for a Complete DMA Session

A DMA session consists of some setup, one or more DMA transfers, and teardown. The host receives an event notification (interrupt) after each DMA transfer (link list consisting of one or more links) completes, and uses this information to define the next transfer unless the information indicates that this was the last transfer in the session. The following steps are involved:

Step 1: Prime the DMA Engine

- Set up an event notification to receive DMA completion interrupts using the SetupEventNotification API.
- When this API returns, the host reads the return parameters for that notification to determine the iTVC15 address and byte count that will be used for the first DMA transfer. Subsequent transfers will be scheduled upon completion of the previous transfer as indicated below.
  - Remember to unmask the appropriate interrupts prior to initiating transfers.

Step 2: Initiate Next DMA Transfer

- There is an event notification (EVENT_DMA_TOHOST_COMPLETION or EVENT_DMA_FROMHOST_COMPLETION) when a DMA transfer completes. The host reads the bits selected during event notification setup in the interrupt status register (0x40) to determine the direction of the DMA transfer just completed. Both bits may be asserted by the firmware if interrupts are pending in both directions. The host may decide either to handle the interrupts sequentially (by servicing and clearing them one at a time), or to handle them at the same time (by servicing both and clearing them during the same ISR).
- The host reads the return parameters of the event notification to learn:
  - Information about the previous transfer (unless this is the first transfer),
  - The Destination information for the next DMA transfer if this is a DMA from Host operation, or
  - The Source information for the next DMA transfer if this is a DMA to Host operation.
- The host uses this information to complete the DMA link list and sends the base address of this list to the firmware using SetDMAFromHostLinkAddress or SetDMAToHostLinkAddress, as appropriate.
  - Normally the host waits until an API returns. However, in this case, to expedite the ISR the host should continue without waiting.
- The host clears the bit(s) in the interrupt status register for which the interrupt was handled.

Step 3: End the DMA Session

- When the DMA status does indicate an end of session, the driver should take the necessary steps to end the DMA session. Events are unregistered using the CancelEventNotification API.

2.5.2.2 Corresponding Actions Taken by the Firmware

The following describes how the firmware responds to the API calls from the host and when it generates events:

- In response to a SetDMAFromHostLinkAddress or SetDMAToHostLinkAddress API call, the firmware programs the DMAC to retrieve the information from the host memory. The DMAC fetches the information and programs its internal link registers to transfer the requested data.
- Once the firmware has programmed the DMAC, it will initiate the DMA transfer immediately for a DMA-from-Host operation. For a DMA-to-Host operation, it will initiate the transfer immediately after the data is available in iTVC15 memory.
- When the firmware has transferred all of the data for the current DMA transfer and is either ready to receive more data, or has enough data for transfer to the host, it sends an event notification to the host to prepare the next transfer.

2.5.2.3 Notes on DMA Usage

The following detail what choices can be made by the driver and what constraints it must satisfy when using the iTVC15 DMAC:

- All DMA transfers must be aligned on a 4-byte boundary.
- DMA transfers to and from the host can be concurrent.
- Each complete link list must consist of transfers for only one stream type, which must match the stream type indicated in the event notification results.
- Note that for a DMA-from-host transaction, the host may choose to send less data than was requested by the firmware in the event notification results, depending on data availability. In this case, the firmware will periodically send additional event notifications to receive the remainder of its original request.

2.5.3 MPEG Streaming Data Port Transfers

2.5.3.1 Encoding

The iTVC15 provides a 256-byte FIFO and control signals to allow hardware to read the data from the iTVC15.

2.5.3.2 Decoding

The iTVC15 provides a 256-byte FIFO and control signals to allow hardware to write the data to the iTVC15.

SECTION 3
OPERATIONAL UNITS

3.1 Encoding

3.1.1 Introduction

There are three distinct steps to be followed to encode using iTVC15. The first two steps must be implemented before the third, but otherwise there are no timing constraints:

1. Configuration of the encoding engine because the iTVC15 can capture and compress data in a number of ways and with different parameter values.
2. Configuration of the audio and video input peripherals to provide appropriately formatted data for the iTVC15 to encode.
3. Run-time control of the encoding session.

3.1.2 Step 1: Program iTVC15 for Encode Mode

Programming the iTVC15 for Encode mode is the first of three steps. The iTVC15 has a video pre-processing unit, a video encoding unit, an audio encoding unit, and a system-level multiplexing unit, all of which must be configured before starting a capture session.

3.1.2.1 Video Input Pre-Processing

The Video Input Module (VIM) of the iTVC15 provides extensive pre-processing. This includes the following capabilities:

- Video Capture
- Noise Reduction
- Inverse Telecine
- VBI Capture

Video Capture

A video capture API, `SetVideoResolution`, is used to set the video size to be captured by the iTVC15. Because any scaling will be done by external hardware, this resolution should account for the scaling. This captured video can be cropped prior to encoding.

Noise Reduction

The noise reduction engine contains three types of filters that can operate together: temporal, spatial, and median. The temporal and spatial filters can operate in a static mode with selectable filter levels and a dynamic mode in which the filter levels are controlled in a fully automated process. The median filter can be set to any combination of horizontal and vertical modes, or disabled altogether.

Inverse Telecine

Inverse telecine is a fully automated process for converting a 25-fps source to 30 fps. By detecting repeated fields, the encoding process can be more efficient by not encoding the duplicate fields.

VBI Capture

Individual lines of pre-sliced VBI can be captured. For each line of interest, these APIs must be called. The external video decoder must also be programmed by the driver to detect and slice the VBI prior to capture by the iTVC15.

3.1.2.2 Video Encoding Parameters

The video encoding engine is very flexible and can be programmed for different input resolutions. Different encoding parameters can be specified. The video resolution defines the source window that will be captured. The specific region that should be encoded is then specified by an API which specifies the "region of interest" which can crop within the source window.

If it is required that the video be scaled (e.g., to process a video signal at less than full resolution [720 pixels/line]), then external hardware must scale the video within the CCIR-656 signal. The video resolution that is specified to the iTVC15 should account for this scaling.

After defining the source, the video bit rate and GOP structure can also be defined.

3.1.2.3 Audio Encoding Parameters

Audio input to the iTVC15 is not pre-processed. Consequently, the hardware must ensure that the audio input matches the specified sampling rate since the iTVC15 does not perform any sample rate conversion internally. Two algorithms are available for audio compression: MPEG Layer II and Dolby Digital (AC-3).

If the audio is to be compressed using the MPEG Audio Layer II algorithm, then the following API should be used:

`SetMPEGAudioEncodingParameters`

3.1.2.4 System Multiplexing Parameters

The compressed data can be multiplexed in various ways for different applications (TS, PS, M1S). It can also be separate audio and video PES packets, but this is only allowed over the PCI interface where these different data types (audio and video) are placed in separate memory areas.

3.1.2.5 Physical Output Port

The data from the iTVC15 encoder can be routed through the PCI interface or the MPEG streaming output data port. The MPEG streaming data port is driven by an external clock and provides data in either continuous mode or handshake mode.

The continuous mode is intended for Transport stream formatted data that is typically used for transmission applications. In this case, the MPEG streaming data port automatically adds null packets, as needed, to match the compressed data rate determined by the external clock.

The handshake MPEG streaming data port mode is intended for Program stream or MPEG-1 system (M1S) formatted data. In this case, the data rate is set by the compressed rate, rather than the external clock.

When the microcontroller interface is used, the data must be output via the streaming data port and there is no need to make this API call.

When the PCI interface is used for control, the data can be routed through the streaming port, or PCI, as selected with this API.

3.1.3 Step 2: Program Audio/Video Input Peripherals

It is necessary to inform the iTVC15 of certain settings that are set via I^2C. This includes the audio sampling rate and the video format and video resolution (after scaling).

3.1.3.1 Audio

The key parameter that must be coordinated here is the audio sampling rate. Depending on whether the audio is to be encoded via MPEG or AC-3, the specific API that selects the sampling rate for the iTVC15 is different. The sampling rate is set in one of the encoding parameter fields set of the following API:

`SetMPEGAudioEncodingParameters`

3.1.3.2 Video

The video parameters that must be coordinated are the frame rate, frame size (after any external scaling), and any VBI capture. The iTVC15 sets frame rate and size using the following two APIs:

`SetVideoFrameRate`
`SetVideoResolution`

3.1.4 Step 3: Control Encoding Session

Controlling the encoding session is the third of the three steps. The encoder can be started and stopped. Certain APIs are also valid during the encode session.

3.1.4.1 Capture Mode

The iTVC15 can operate in multiple modes:

- **MPEG Capture.** This mode is used for normal audio/video capture and compression with optional VBI capture as defined in the preceding APIs.
- **Audio Only Capture.** This mode is used for time-shifting radio and other applications that have no video requirement. In this case the video encoder is not started and the audio parameters are set with the APIs defined earlier.
- **VBI Only Capture.** This mode is used for data broadcasting applications. In this case, the audio and video encoders are not started. Instead up to the full of VBI data is captured.

3.1.4.2 Starting and Stopping

After establishing the capture mode, a capture session is started using `StartCapture`. `StopCapture` stops the current capture.

3.1.5 Event Notifications

Please refer to Section 2.4 for details concerning event notification.

3.2 Decoding

3.2.1 Introduction

Since the iTVC15 does not necessarily control the configuration of the audio/video peripherals, it is important that the iTVC15 and the host work together smoothly to effectuate any changes that might be required when switching between two differently encoded bitstreams. This might occur when the user switches between two channels that have been encoded with different audio sampling rates. The other potential issue would be when switching between NTSC and PAL formats, although this is unlikely to happen in a production environment.

There are three distinct steps to be followed to decode using iTVC15:

1. Configure the decoding engine because the iTVC15 can "play" audio and video data that are stored in a number of different formats.
2. Configure the audio and video output peripherals, as necessary, to accept the formatted uncompressed data output from iTVC15.
3. Implement run-time control of the decoding session until there is a "channel change" and the bitstream changes which may necessitate reconfiguration of the iTVC15 and/or the audio/video peripherals.

3.2.2 Step 1: Program iTVC15 for Decode Mode

Programming the iTVC15 for Decode mode is the first of three steps.

3.2.2.1 Physical Input Port

The data input to the iTVC15 is through the PCI interface or MPEG streaming data port. The MPEG streaming data port is driven by an external clock and provides data in either continuous or handshake modes. The continuous mode is intended for TS formatted data and is used in transmission applications. Handshake mode is intended for PS or M1S formatted data.

When the microcontroller interface is used, the data must be output via the streaming data port and there is no need to make this API call.

When the PCI interface is used for control, the data can be routed through the streaming port, or PCI, as selected with this API.

3.2.2.2 MPEG Format

As already indicated, the iTVC15 decoder can decode compressed data that has been multiplexed in various ways by different applications (TS, PS, M1S). It can also decode raw (non-packetized) elementary streams that contain no timing information as well as packetized, but not multiplexed, PES packets. However, ES and PES are only allowed over the PCI interface where these different data types (audio and video) are placed in separate memory areas. Note that ES does not contain any time stamps, so there is no attempt to synchronize audio and video.

The iTVC15 decoder can also "play" pulse code modulation (PCM) audio that has not been compressed. This mode is similar in function to audio in an ES format and, in this mode, there is also no attempt to synchronize audio and video.

```
Mode:          audio_only_PCM
               audio_only ES
               audio_only PES
               video_only_ES
               video_only_PES
               audio_&_video_ES
               audio_PCM_&_video_ES
               audio_&_video_PES
               system_MPEG1
```

```
system_PS
system_TS
```

3.2.3 Step 2: Program Audio/Video Output Peripherals

Programming the audio/video output peripherals is the second of the three steps. The information to configure the peripherals (audio sampling rate, TV format) is contained in the bitstream itself (with the exception of PCM audio, which is not an MPEG standard format).

There are two methods for configuring the audio/video peripherals before the first stream is decoded, or reconfiguring each time there is a channel change resulting in a "ragged" bitstream change. One lets the iTVC15 do all of the parsing, which is simplest for the driver but has the largest latency. The second minimizes the latency by having the driver parse the bitstream.

3.2.3.1 I^2C Controller

For those peripherals that are controlled via I^2C, there are two hardware solutions. The first assumes a customer-provided controller. Alternatively, the iTVC15 I^2C controller can be used. If you wish to use the iTVC15 I^2C controller, you can access individual registers on the I^2C control bus using the provided source code as a reference.

3.2.3.2 Configure Video Encoder for VBI (Optional)

Depending on type of VBI and video encoder, the actual data will be sent via I^2C or a serial data pin. In any case, the encoder should be initialized as necessary.

3.2.4 Step 3: Control Decoding Session

Controlling the decoding session is the third of the three steps. The decoder can be started and stopped. Certain APIs are also valid during the decode session. These are listed below.

3.2.4.1 Audio/Video Playback Control

Start

`StartPlayback`

This API is only needed when a NewChannel API has been called with "handshake" set to "yes."

Stop

`StopPlayback`

After this API, the sequence header information persists for the next stream until a new sequence header is parsed.

Pause

`PausePlayback`

Pause stalls the parsing of the decoder, mutes the audio, and freezes the video. Depending on the mode, one or two video fields are displayed during freeze.

Fast Forward

`SetTrickMode`

Fast playback is accomplished by only decoding I/P or I-pictures as specified via this API.

Slow Motion

`SetTrickMode`

Step

`Step`

Step forward 1 to 30 frames.

3.2.4.2 Graphics Control

The iTVC15 graphics subsystem can be used to compose high-quality graphics at high speed to support animation without "tearing" using double-buffered techniques. The video can be scaled and placed "within" the graphics or the OSD graphics can be placed "within" the video picture using each pixel's alpha value, or a global alpha value.

OSD Sub-system

The OSD engine is mainly designed to overlay RGB graphics on top of decoded YCbCr video. Examples of graphics data include on-screen channel guide, video playback status, "buttons", and icons. The graphical data within the OSD buffer is first flicker-filtered, then mixed with the main picture display.

However, the iTVC15 OSD can also support Web browsing applications where a virtual Web page is larger than the physical display. Hardware controls for panning and scrolling the on-screen content are provided including vertical wraparound.

Typically, there are two full-screen OSD buffers, and one is being displayed while the other is being composed. An additional part of memory can be used to cache those text/graphics that will be used for subsequent compositions.

The actual OSD buffer usage model is fully determined by driver actions using the APIs described below.

The first step is to define the OSD buffers and any other buffers that are to be used as scratch area for text and graphics. As described later, buffer areas for BLT command lists are also allocated in this contiguous "Graphics" buffer area on the iTVC15.

Before defining the buffers, the driver must query the iTVC15 to determine how much memory is available, and where it is located, for all of these buffers via the following call:

`GetGraphicsBufferSpace`

The driver then determines how best to utilize this memory and informs the iTVC15 of the pixel format that will be used for OSD. The corresponding depth is either 8 bits for color-indexed pixels (CLUT-8) or 32 bits for true color plus alpha (ARGB 8:8:8:8). In the case of 32-bit deep OSD there is a special source format of 8-bit alpha that can be used for efficiently storing fonts. This font is expanded to 32 bits by the BLT engine to fill the OSD buffer before rendering as described below (`BLTExpandText`).

`Set/GetOSDPixelFormat`

The two supported formats are shown below.

CLUT-8

bits 31–24	bits 23–16	bits 15–8	bits 7–0
Pix 3	Pix 2	Pix 1	Pix 0

ARGB 8:8:8:8

bits 31–24	bits 23–16	bits 15–8	bits 7–0
Pix A	Pix R	Pix G	Pix B

The CLUT index is in the range 0–255 and for each index a full 32-bit ARGB 8:8:8:8 pixel is defined. Color keying can be achieved by setting the alpha value to 0 for the color index value that would normally be consider the color key value. This will show the main picture pixels (video) wherever the OSD buffer contains this index value.

Alpha Value

Alpha is used as a blend factor between the OSD and Main Picture for effects such as fade and translucency. Alpha can be used to "crop" OSD to random shapes. The OSD logic supports Local and Global Alpha.

Local alpha is an 8-bit value that is part of each pixel or, in the case of CLUT-8, part of the data value in the Color LUT (CLUT).

Global Alpha is a simple way to alter alpha for the entire OSD region without requiring that each pixel's alpha value be modified. The global alpha (when enabled) is blended with each pixel's local alpha. The resulting value is used as the pixel's alpha blend value with the corresponding main picture pixels.

The OSD 8-bit global alpha can be set by the following API:

Set/GetGlobalAlpha

The driver computes the base address of each OSD buffer that will be used and based on the size of the buffer calculates whether a single or multiple OSD buffers will be used in the specific implementation. The OSD buffer size should not be less than the physical display window size. If it is larger, then the driver can pan and scan the OSD buffer within the display window.

The display window is the actual location of the graphics and text from the OSD buffer on the screen. The screen coordinate system uses X to indicate horizontal position with 0 indicating the left-most pixel and 719 the right-most. Y indicates the line number with 0 being the top line and 479 (or 575) being the last for 525/60 (NTSC) and 625/50 (PAL) systems, respectively.

The window is positioned using the following API:

Set/GetOSDScreenCoordinates

By default the OSD is disabled. It is enabled by specifying a buffer for display using the following API:

Set/GetOSDDisplayedBuffer

The width indicates the stride of the OSD buffer in pixels and the height is the number of lines in the OSD buffer. The base address must be dword aligned and the width/height must be no less than the screen window width/height for proper operation. Positive offset parameters are used for positioning the OSD buffer on the screen when the OSD buffer is larger than the screen window.

The OSD can be disabled at any time by a call to the following API:

Set/GetOSDState

If the OSD buffer is larger than the OSD, then the region from the OSD buffer that is displayed can be specified to allow panning and scrolling of the OSD buffer on screen. The OSD should not be panned beyond the edges of the OSD buffer horizontally. However, the OSD buffer will wrap vertically after the last line is displayed, by continuing from the top of the OSD buffer to support scrolling. Initial display offsets are specified in **SetOSDDisplayedBuffer** subsequently they are controlled via the following API:

MoveOSDBuffer

The OSD buffer can be filtered, prior to display, by the flicker reduction module. This is controlled by the following API:

Set/GetOSDDeflickerState

Composing the OSD via BLT

The BLT engine is designed to accelerate composing graphics in the OSD. The BLT hardware performs memory-intensive tasks more efficiently than firmware because it accesses memory at speed. In addition, memory operations can take place concurrently with firmware execution of other tasks.

Memory Interface

BLT access to and from memory is done 32 bits at a time.

Basic Operation

The basic operation the BLT engine performs is copying a source region to a destination region. Both regions are required to be the same width and height. It is required that the source and destination be non-overlapping.

Source/Destination Memory Addressing

Both source and destination are rectangular regions in memory. The destination is likely to be an OSD buffer, while the source can be scratch memory containing text or graphics. Each rectangle is not required to be contiguous in memory. For each region, there is a fixed size "gap" between the end of one line and the start of the next. The offset between the start of one line and the next is the stride. For flexibility, source and destination regions have independent strides.

Data Type

32-bit OSD. When the OSD pixel format is set to ARGB 8:8:8:8, there are two supported source formats: Alpha-8, which is intended for text rendering, and 32-bpp ARGB, which is intended for graphics. The destination format is 32-bpp ARGB in all cases and an alpha color expansion mode is supported in hardware to accomplish this. The pixel organization in memory is shown below.

Alpha-8

bits 31–24	bits 23–16	bits 15–8	bits 7–0
Pix 3	Pix 2	Pix 1	Pix 0

ARGB 8:8:8:8

bits 31–24	bits 23–16	bits 15–8	bits 7–0
Pix A	Pix R	Pix G	Pix B

8-bit OSD. When the OSD pixel format is set to CLUT-8 (see earlier API) there is only one supported source format, CLUT-8. The destination format is also CLUT-8 and the pixel organization in memory is explained below.

CLUT-8

bits 31–24	bits 23–16	bits 15–8	bits 7–0
Pix 3	Pix 2	Pix 1	Pix 0

Operation Memory Alignment

It is required that the start of each line in the source rectangle is 32-bit aligned to match the memory interface width. The destination rectangle must have pixel alignment. Depending on the OSD format (8 or 32 bits), the destination line must be 8- or 32-bit aligned.

BLT Commands

The BLT engine supports copying graphical data into the OSD. The BLT copies a source rectangle to a destination rectangle located at a different address. The source and destination rectangles are restricted to be non-overlapping. The following API is provided:

`BLTCopy`

The BLT engine can be set to fill the destination rectangle with a programmed color. The Color Fill operation can be used to preset text background in the OSD. The following API is provided:

`BLTFillColor`

When the OSD pixel format is 32-bit ARGB, the BLT engine supports composing text characters in the OSD, where each pixel in the font is stored as an 8-bit alpha. Alpha-Color expansion enables transparency and edge anti-aliasing, by setting the alpha of each pixel to the proper value.

It is required that the start of each line of font is 32-bit aligned in memory.

The BLT engine blends the programmed text color and the destination pixel color, with the pixel alpha as the blend factor using the following API:

`BLTExpandText`

BLT Engine Control

The driver can use a synchronous mode where each BLT command is issued separately, and the driver must wait until that BLT command completes before issuing the next BLT command. In this mode, the BLT command will execute without any additional driver involvement.

For improved performance, individual BLT commands can be combined into a “BLT command list” that is stored internal to the iTVC15 to improve BLT engine utilization. In this case, the driver software builds the list and then passes it to the iTVC15 which can be programmed, by the driver, to execute the list of BLT commands.

There are no special markers in the list and each list entry is up to 28 bytes in length. Each list entry starts 28 bytes after the previous entry, even if the data required to define that entry is less than 28 bytes. BLTCopy commands require 24 bytes, **`BLTColorFill`** commands require 22 bytes and **`BLTExpandText`** commands require 28 bytes. In this way a list can be recycled by substituting unlike commands.

Each list consists of a modified ROP parameter command that incorporates coding for the specific type of BLT command in the high 8-bits of the 16-bit ROP value. The remaining bytes are just the remaining parameters that would have been passed to the specific BLT command.

To prevent “tearing,” the driver will typically start the list when the destination buffer is off screen.

After directly loading the BLT command list into the graphic space of the iTVC15, the driver uses the following API when it would like the command list to be processed. Note that the base address of the command list must be 32-bit aligned.

Completion Interrupt. To support non-polling software, each BLT operation can be programmed to generate an interrupt upon completion. When a BLT list is used, the last BLT can be set to generate an “end-of-list” interrupt.

3.2.4.3 Video Output Control

When decoding MPEG streams, the iTVC15 can change its output aspect ratio to support 4x3 and 16x9 material in normal and letterbox modes using the following API:

`SetDisplayMode`

The iTVC15 provides hardware for scaling the decoded MPEG picture up or down. Certain limitations apply:

- **Up-scaling:** up to 400%, final size no larger than 720x576.
- **Down-scaling:** down to 25%, original size no larger than 720x576.

When the scaled picture is less than full-screen, the output video window can be positioned anywhere on the screen that it is fully visible. In other words, the video window cannot be "pushed" off the screen.

The scaling and positioning are all controlled through the following API:

`SetVideoScreenCoordinates`

where the positioning (offset_x, offset_y) is relative to a default top left justification.

3.2.4.4 Event Notifications

Please refer to Section 2.4 for details.

3.3 Trick Modes APIs

3.3.1 Decoder

The iTVC15 firmware provides a flexible interface for the implementation of trick modes, which allows for playback speed changes in both directions. The firmware supports a frame accurate mechanism for playback speed changes between 1/16X normal speed to 2X normal speed. For playback speeds higher than 2X in either direction, the firmware provides a flexible interface to the host, giving it complete control over the speed and accuracy by throttling the data transfer to the firmware.

The trick modes are implemented through firmware event notifications. The basic steps are as follows:

1. The host estimates the amount of data, which needs to be periodically skipped, to reach the desired playback speed. Also, it estimates the periodic delay (if any) required to reach the desired speed.
2. The host initiates the desired trick mode through the API call **`SetTrickMode`** (at this point, the decoder immediately enters the paused state and returns any pending event notifications). With this call, the host determines the direction and speed of playback, as well as the temporal reference of the last picture in a GOP to be considered for presentation by the decoder.
3. The host continues to send data until the firmware generates a DMA completion event notification with the status flag set to **`PROP_DMA_STATUS_ENDOFSEGMENT`** indicating that the decoder has decoded all pictures (of the selected types) dependant on the last received I frame, and is ready to receive a new segment starting with an I frame.
4. The host seeks to a new location in the bit stream, bypassing the amount of data estimated in step 1. The host also waits for the duration estimated in step 1. For continuous playback, this is a seek to the previous or next I frame (data may begin with an incomplete frame immediately before the I frame). In the forward direction, playback begins with the first frame in display order (of the selected types), for which an I frame is available. In the reverse direction, playback begins with the frame (of the selected types) having the highest temporal reference equal to or lesser than the maximum temporal reference value set when trick mode is initiated.
5. Repeat steps 3 through 4 until the user changes the playback speed or returns to normal playback. At this point, the host terminates the trick mode by calling **`SetTrickMode`** again with **`PROP_TRICKMODE_SPD_NORMAL.`** All pending event notifications are returned.

Note: The hardware was designed to run at certain frame rate, and so we can't arbitrarily increase the playback speed. Without the assistance of host (which can parse and drop some data for faster playbacks), a maximum of ~2X forward speed is possible. Due to additional overhead, the reverse speed in fine mode may be even less.

3.3.2 Encoder

iTVC15 also provides information about location of frames in the bitstream in order to assist in random access when the streams are played back. The firmware generates a table, in memory, of a set of values for each frame, which is updated in a circular fashion.

The host is expected to remove the data at an appropriate pace so that the firmware does not overwrite critical information. The presentation time stamp (PTS) is provided so that the host may synchronize to the correct picture information if data loss occurs.

A write location pointer is also provided to assist the host in determining the availability of data. This pointer is placed immediately before the table in memory and it indicates the actual table entry (in number of entries from the top of the table) where the firmware is writing new information. Write location pointers are updated with each picture that is encoded and multiplexed.

The host has the option of changing the picture types entered into the table, as well as the depth of the table by calling **`IndexTblParams.`** This API must be called prior to starting a capture session.

Table 3-1: Memory Allocation

<table>
<tr><th></th><th>4 Bytes</th><th>8 Bytes</th><th>4 Bytes</th><th>4 Bytes</th><th>4 Bytes</th></tr>
<tr><td>Initial</td><td>Reserved</td><td>Reserved</td><td>Reserved</td><td>Reserved</td><td>Writer Pointer</td></tr>
<tr><td>Pic1</td><td rowspan="4">Total # of bytes for the multiplexed stream containing the full picture (includes pack/ packet headers and possibly packets with other stream types)</td><td rowspan="4">Byte offset from beginning of encode session to the first byte of the pack/packet containing the picture_start_code</td><td rowspan="4">Picture type</td><td colspan="2" rowspan="4">PTS from the first packet containing the picture_start_code</td></tr>
<tr><td>Pic 2</td></tr>
<tr><td>...</td></tr>
<tr><td>Pic N</td></tr>
</table>

SECTION 4
EVENT NOTIFICATIONS

4.1 Introduction

This section details the individual event notifications that can be received from the firmware. Each event notification is described using the format illustrated below. Note that the Event Notification Name is an input parameter to the SetupEventNotification API (described in Section 5) that is used to "hook" an event.

Each command is described using the following format:

EVENT NOTIFICATION NAME

Description

Event description.

Return Values

Description of the values (R1-R16) returned by this event.

4.2 Event Notification Commands Summary Table

Table 4-1: Event Notification Commands Summary Table

Event Notification Grouping	Event Notification Command	Page
Common	EVENT_ERROR_DETECTED	4-2
DMA Event	EVENT_DMA_TOHOST_COMPLETION	4-3
	EVENT_DMA_FROMHOST_COMPLETION	4-4
Encoder	EVENT_VIDEO_INPUT_VSYNC	4-5
Decoder	EVENT_VIDEO_OUTPUT_VSYNC	4-6
	EVENT_VIDEO_PICTURE_DECODED	4-7
	EVENT_VIDEO_PICTURE_SKIPPED	4-8
	EVENT_VIDEO_SEQUENCE_END_CODE	4-9

4.3 Common Event Notifications

4.3.1 EVENT_ERROR_DETECTED

Description

This event notification generates an interrupt to the host each time an error is detected by the firmware.

Return Values

Parameter	Description
R1	Status: EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise
R2	A 4-byte value indicating the type of error.

4.4 DMA Event Notifications

4.4.1 EVENT_DMA_TOHOST_COMPLETION

Description

This event notification generates an interrupt to the host each time a DMA transfer to the host completes.

Return Values

Parameter	Description
R1	Status: EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise
Information Regarding the Previous Transfer	
R2	Status of previous DMA transfer: PROP_DMA_STATUS_INPROGRESS PROP_DMA_STATUS_DONE PROP_DMA_STATUS_ENDOFSTREAM PROP_DMA_FAILURE (iTVC15 failed and must be reset)
R3	Number of bytes transferred.
Information Regarding the Next Transfer	
R4	Stream type for the next DMA transfer: PROP_STREAMTYPE_MPEGMULTIPLEX PROP_STREAMTYPE_VIDEOPES PROP_STREAMTYPE_AUDIOPES PROP_STREAMTYPE_AUXPES
R5	Byte offset in iTVC15 memory where the data should be transferred.
R6	Number of bytes that are ready for transfer from the iTVC15 memory.

4.4.2 EVENT_DMA_FROMHOST_COMPLETION

Description

This event notification generates an interrupt to the host each time a DMA transfer from the host completes.

Return Values

Parameter	Description
R1	Status EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise
Information Regarding the Previous Transfer	
R2	Status of previous DMA transfer: PROP_DMA_STATUS_INPROGRESS PROP_DMA_STATUS_DONE PROP_DMA_STATUS_ENDOFSTREAM PROP_DMA_STATUS_ENDOFSEGMENT PROP_DMA_FAILURE (iTVC15 failed and must be reset)
R3	Number of bytes transferred.
Information Regarding the Next Transfer	
R4	Stream type for the next DMA transfer: PROP_STREAMTYPE_MPEGMULTIPLEX PROP_STREAMTYPE_VIDEOPES PROP_STREAMTYPE_AUDIOPES PROP_STREAMTYPE_AUXPES
R5	Byte offset in iTVC15 memory where the data should be transferred.
R6	Number of bytes that can be overwritten in the iTVC15 memory.

4.5 Encoder Event Notifications

4.5.1 EVENT_VIDEO_INPUT_VSYNC

Description

This event notification generates an interrupt to the host at each vertical sync interrupt received by the VIM within the iTVC15.

Return Values

Parameter	Description
R1	Status EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise

4.6 Decoder Event Notifications

4.6.1 EVENT_VIDEO_OUTPUT_VSYNC

Description

This event notification generates an interrupt to the host at each vertical sync interrupt received by the display engine within the iTVC15.

Return Values

Parameter	Description
R1	Status EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise

4.6.2 EVENT_VIDEO_PICTURE_DECODED

Description

This event notification generates an interrupt to the host after each complete field or frame picture is successfully decoded and displayed by the MPEG decoder.

Return Values

Parameter	Description
R1	Status: EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise
R2	Type of decoded picture: PROP_PICTURE_FIELD_PICTURE PROP_PICTURE_FRAME_PICTURE
R3	Coding type of decoded picture: PROP_PICTURE_INTRA PROP_PICTURE_PREDICTED PROP_PICTURE_BIDIRECTIONAL PROP_PICTURE_DCINTRA

4.6.3 EVENT_VIDEO_PICTURE_SKIPPED

Description

This event notification generates an interrupt to the host each time a complete field or frame picture is discarded by the MPEG decoder.

Return Values

Parameter	Description
R1	Status: EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise
R2	Type of skipped picture: PROP_PICTURE_FIELD_PICTURE PROP_PICTURE_FRAME_PICTURE
R3	Coding type of skipped picture: PROP_PICTURE_INTRA PROP_PICTURE_PREDICTED PROP_PICTURE_BIDIRECTIONAL PROP_PICTURE_DCINTRA

4.6.4 EVENT_VIDEO_SEQUENCE_END_CODE

Description

This event notification generates an interrupt to the host each time a sequence end code is detected by the MPEG decoder.

Return Values

Parameter	Description
R1	Status EN_RC_SUCCESS if the event completes within a specific time; EN_RC_ERROR_UNKNOWN otherwise

SECTION 5
API COMMANDS

5.1 API Command Description Format

This section details the individual API commands. Each command is described using the following format:

API Command Name

Synopsis and Command Code

Enumerated value as defined in doapi.h.

Description

Functional description.

Input Parameters

Descriptions of any input parameters (P1-P16).

Return Values

Descriptions of any return values (R1-R16).

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2 Encoder Commands

This section details those commands that are specific to an encoding session, including:

Table 5-1: Encoder Commands Summary Table

5.2.1 GetDataXferStatus

Synopsis and Command Code

0xC6 (198)
ApiCommand_Encoder_GetDataXferStatus

Description

This command gets the encoder buffer information. This command extracts the sequence end code. After sending StartCapture, the firmware still generates a few interrupts. The last of these interrupts has R1 = = 1 and a partial buffer to copy of size R2.

Input Parameters

None.

Return Values

Parameter	Description
R1	Transfer state: '1' if this is the last buffer.
R2	Size of the last buffer. Valid only if R1 == 1.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.2 GetDMAToHostTransferInfo

Synopsis and Command Code

0xCA (202)
APICommand_Encoder_GetDMAToHostTransferInfo

Description

This command queries the status of the previous transfer and determines whether this is a DMA-to-Host or a DMA-from-Host operation. The DMA types are defined in the DMA_TYPE_ENCODER enum in ivactypes.h.

This command is provided as a static mailbox along with the DMA done interrupt (bit 27). This command is provided in mailbox #10.

Input Parameters

None.

Return Values

Parameter	Description
R1	Stream Type
R2	Offset Address
R3	Maximum amount of transfer

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.3 GetDMATransferStatus

Synopsis and Command Code

0xCB (203)
APICommand_Encoder_GetDMATransferStatus

Description

This command queries the DMA controller for the status of the last operation. The DMA types are defined in the DMA_TYPE_ENCODER enum in ivactypes.h.

This command is provided as a static mailbox along with the DMA done interrupt (bit 27). This command is provided in mailbox #9.

Input Parameters

None.

Return Values

Parameter	Description
R1	DMA Transfer Status: Bit 0 is set to '1' if transfer is completed. Bit 2 is set to '1' if there are any DMA transfer errors. Bit 4 is set to '1' if the link list has any errors.
R2	DMA Type
R3	PTS bits 31–0
R4	PTS bit 32

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.4 GetEncoderFWVersion

Synopsis and Command Code

`0xC4 (196)`
`ApiCommand_Encoder_GetEncoderFWVersion`

Description

This command gets the encoder firmware version information.

Input Parameters

None.

Return Values

Parameter	Description
R1	Firmware version. The structure is 0xXXYYZZZZ, where XX represents the major version number, YY represent the minor version number, and ZZZZ represent the firmware build.

Return Codes

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.5 IndexTblParams

Synopsis and Command Code

`0xC7 (199)`
`APICommand_ Encoder_IndexTblParams`

Description

This command sets the parameters of the Index Table. Please refer to the "Trick Modes" section of the *iTVC15 Host Driver Development Reference* for more details.

Input Parameters

Parameter	Description
P1	Picture mask. Allowed values are: 0: Disable the index capture 1: I-frames only 3: I- and P-frames 7: All frames (default)
P2	Number of requested elements (maximum of 400).

Return Values

Parameter	Description
R1	Table position (offset of the table in SDF memory).
R2	Number of elements actually allocated (maximum P1).

Return Code

`API_RC_SUCCESS` if the function succeeds;
`API_RC_ERROR_UNKNOWN` otherwise

5.2.6 InputInit

Synopsis and Command Code

`0xCD (205)`
`APICommand_ Encoder_InputInit`

Description

This command initializes the digital video input to the iTVC15.

Input Parameters

None.

Return Values

None.

Return Code

`API_RC_SUCCESS` if the function succeeds;
`API_RC_ERROR_UNKNOWN` otherwise

5.2.7 InputRefresh

Synopsis and Command Code

0xD3 (211)
APICommand_ Encoder_InputRefresh

Description

This command refreshes the digital video input to the iTVC15.

Input Parameters

None.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds;
API_RC_ERROR_UNKNOWN otherwise

5.2.8 NoOp

Synopsis and Command Code

0x80 (128)
APICommand_Encoder_NoOp

Description

This command performs no operation. Usually it is used to check whether the firmware is alive.

Input Parameters

None.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.9 PauseResumeEncode

Synopsis and Command Code

0xD2 (210)
APICommand_ Encoder_PauseResumeEncode

Description

This command resumes encoding after a pause. When the encoder is paused, it drops all frames without encoding.

Input Parameters

Parameter	Description
P1	Pause/Resume: 0: Pause 1: Resume

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds;
API_RC_ERROR_UNKNOWN otherwise

5.2.10 ProvidePlaceHolder

Synopsis and Command Code

0xD7 (215)
APICommand_ Encoder_ProvidePlaceHolder

Description

This command allows for the addition of a placeholder for proprietary data.

Input Parameters

Parameter	Description
P1	Type: 0: extension_and_user_data 1: private back 1 (stream ID is 0xBD)
P2	Period at which to insert the data (new packet to be inserted every P2 units). The units are as follows: extension_and_user_data and DVD streams -> unit = GOP (15 frames for GOP 15) Private packet -> unit = frame
P3	Size of the data to be inserted in DWORDs (if the value is more than 9 DWORDs, P4 to P12).
P4	DWORD1 to be inserted.
P5	DWORD2 to be inserted.
P6	DWORD3 to be inserted.
P7	DWORD4 to be inserted.
P8	DWORD5 to be inserted.
P9	DWORD6 to be inserted.
P10	DWORD7 to be inserted.
P11	DWORD8 to be inserted.
P12	DWORD9 to be inserted.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds;
API_RC_ERROR_UNKNOWN otherwise

5.2.11 SetAspectRatio

Synopsis and Command Code

0x99 (153)
APICommand_Encoder_SetAspectRatio

Description

This command sets the aspect ratio for encoded pictures. The actual change takes place at the end of the current encoding GOP.

Input Parameters

Parameter	Description
P1	4-bit aspect ratio information as defined in the MPEG specification.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.12 SetAudioPID

Synopsis and Command Code

0x89 (137)
APICommand_Encoder_SetAudioPID

Description

This command sets the audio PID for Transport streams.

Input Parameters

Parameter	Description
P1	Audio PID

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.13 SetAuxPID

Synopsis and Command Code

0x8d (141)
APICommand_Encoder_SetAuxPID

Description

This command sets the PID for the PCR packets for Transport streams.

Input Parameters

Parameter	Description
P1	Aux PID

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.14 SetCaptureLineNo

Synopsis and Command Code

0xD6 (214)
APICommand_Encoder_SetCaptureLineNo

Description

This command sets the line count for field1 and field2 of the analog video decoder.

Input Parameters

Parameter	Description
P1	Line count for field1: 0xEF for Philips SAA7114 0xF0 for Philips SAA7115 0x105 for Micronas
P2	Line count for field2: 0xEF for Philips SAA7114 0xF0 for Philips SAA7115 0x106 for Micronas

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.15 SetClosedGOP

Synopsis and Command Code

0xC5 (197)
ApiCommand_Encoder_SetClosedGOP

Description

This command sets the closed GOP function.

Input Parameters

Parameter	Description
P1	Closed GOP: 0: Off 1: On

Return Values

None.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.16 SetCopyright

Synopsis and Command Code

`0xD4 (212)`

`ApiCommand_Encoder_SetCopyright`

Description

This command sets the copyright for the encoded bitstream.

Input Parameters

Parameter	Description
P1	Copyright: 0: No copyright protection 1: Protected by copyright

Return Values

None.

Return Codes

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.17 SetDMABlockSize

Synopsis and Command Code

0xC9 (201)
ApiCommand_Encoder_SetDMABlockSize

Description

This command sets the transfer size for every DMA access. This size can be overridden when actually issuing the DMA transfer command.

Input Parameters

Parameter	Description
P1	Block size in bytes for every DMA transfer.

Return Values

None.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.18 SetDMAToHostLinkAddress

Synopsis and Command Code

`0xCC (204)`
`APICommand_Encoder_SetDMAToHostLinkAddress`

Description

This command programs the DMA controller for a DMA-to-Host operation.

Input Parameters

Parameter	Description
P1	Physical address of the link list
P2	Size of data in the list
P3	DMA type. The DMA types are defined in the DMA_TYPE_ENCODER enum in ivactypes.h, as follows: 0 = MPEG

Return Values

None.

Return Code

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.19 SetDnrMedianCoring

Synopsis and Command Code

0x9F (159)
ApiCommand_Encoder_SetDnrMedianCoring

Description

This command sets coring levels/thresholds for the median filter.

Input Parameters

Parameter	Description
P1	Luma high. This is the threshold above which the median filter is enabled for luma, from 0 to 255. The default value is 0.
P2	Luma low. This is the threshold below which the median filter is enabled for luma, from 0 to 255. The default value is 255.
P3	Chroma high. This is the threshold above which the median filter is enabled for chroma, from 0 to 255. The default value is 0.
P4	Chroma low. This is the threshold below which the median filter is enabled for chroma, from 0 to 255. The default value is 255.

Return Values

None.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.20 SetDnrMode

Synopsis and Command Code

`0x9B (155)`
`ApiCommand_Encoder_SetDnrMode`

Description

This command sets the operating mode of DNR preprocessor.

Input Parameters

Parameter	Description
P1	DNR Mode: 0: Static temporal, static spatial 1: Static temporal, dynamic spatial 2: Dynamic temporal, static spatial 3: Dynamic temporal, dynamic spatial The default mode is dynamic spatial and temporal.
P2	Median Filter: 0: Disable 1: Horizontal 2: Vertical 3: Horizontal / Vertical 4: Diagonal The default median filter mode is horizontal/vertical enabled.

Return Values

None.

Return Codes

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.21 SetEventNotification

Synopsis and Command Code

```
APICommand_Encoder_SetupEventNotification
```

Description

This command enables notification from the firmware to the host for a specific event. The host needs to unmask the interrupt bit. Please refer to the "Data Transfer" section of the *iTVC15 Host Driver Development Reference* for more information.

Input Parameters

Parameter	Description
P1	Event type (one of the values in the enum FWAPI_PARAM_EncoderEventType. The allowed values are as follows: 0 = ENCODER_REFRESH_INPUT
P2	Setup Event Notification State: 0 = Off 1 = On
P3	Interrupt bit assigned to the bit.
P4	Mailbox slot assigned to the event. A value of –1 means that this event does not require a mailbox slot.

Return Values

None.

Return Code

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.22 SetGOPStructure

Synopsis and Command Code

0x97 (151)
APICommand_Encoder_SetGOPStructure

Description

This command sets the GOP structure.

Input Parameters

Parameter	Description
P1	GOP Size
P2	Number of B-frames between I and P+1

Example: For GOP Structure IBBPBB, P1 = 6 and P2 = 3.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.23 SetInverseTelecine

Synopsis and Command Code

`0xB1 (177)`

`ApiCommand_Encoder_SetInverseTelecine`

Description

This commands sets the 3:2 pulldown.

Input Parameters

Parameter	Description
P1	Set Inverse Telecine: 0: Off 1: On

Return Values

None.

Return Codes

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.24 SetMPEGAudioEncodingParameters

Synopsis and Command Code

`0xBD (189)`
`APICommand_Encoder_SetMPEGAudioEncodingParameters`

Description

This command sets the MPEG audio encoding parameters. This command can be called during streaming to change the audio mode "on the fly."

Input Parameters

Parameter	Description
P1	Bitmask specifying the audio parameters, based on the union FWAPI_PARAM_MPEGAudioEncodingParameters in ivacTypes.h: Bits 1–0 — Sampling frequency 00: 44.1 KHz 01: 48 KHz 10: 32 KHz Bits 3–2 — Layer 01: Layer 1 10: Layer 2 Bits 7–4 — Bit rate. This is an index into the bit rate table — check MPEG audio specification. Bits 9–8 — Mode 00: Stereo 01: Joint Stereo 10: Dual 11: Mono Bits 11–10 — Mode extension Bits 13–12 — Emphasis 00: None 01: 50/15 11: CCITT J.17 Bit 14 — Error protection (turns on CRC) Bit 15 — Copyright Bit 16 — Copy/Original

Return Values

None.

Return Code

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.25 SetSkipInputFrame

Synopsis and Command Code

0xD0 (208)
APICommand_ Encoder_SetSkipInputFrame

Description

This command sets the number of input frames to be skipped for each captured frame.

Input Parameters

Parameter	Description
P1	Number of input frames to be skipped for each captured frame. The actual frame would be Rate / (1 + P1), where Rate = 29.97 fps for NTSC or 25 fps for PAL.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds;
API_RC_ERROR_UNKNOWN otherwise

5.2.26 SetSpatialFilterType

Synopsis and Command Code

`0xA1 (161)`
`ApiCommand_Encoder_SetSpatialFilterType`

Description

This command sets the type of spatial pre-filter type.

Input Parameters

Parameter	Description
P1	Luma type: 0: Disable 1: 1D Horizontal Only 2: 1D Vertical Only 3: 2D H/V Separable 4: 2D Symmetric Non-Separable The default type is 2D H/V separable.
P2	Chroma type: 0: Disable 1: 1D Horizontal Only The default type is 1D horizontal only.

Return Values

None.

Return Codes

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.27 SetStaticDnrParams

Synopsis and Command Code

0x9D (157)
ApiCommand_Encoder_SetStaticDnrParams

Description

This command sets filtering levels for static spatial and temporal DNR filters. Effective only when static spatial and/or temporal operating mode is selected.

Input Parameters

Parameter	Description
P1	Spatial. Filter level from 0 to 15, inclusive. The default value is 0, effective only when static spatial operating mode is selected.
P2	Temporal. Filter level from 0 to 31, inclusive. The default value is 0, effective only when static temporal operating mode is selected.

Return Values

None.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.28 SetStreamOutputPort

Synopsis and Command Code

0xBB (187)
APICommand_Encoder_SetStreamOutputPort

Description

This command sets the stream output port.

Input Parameters

Parameter	Description
P1	Stream Output Port: 0: Memory (default) 1: Streaming 2: Serial

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.29 SetStreamOutputType

Synopsis and Command Code

0xB9 (185)
APICommand_Encoder_SetStreamOutputType

Description

This command sets the stream output type.

Input Parameters

Parameter	Description
P1	The stream output type is one of the following: 0: Program stream 1: Transport stream 2: MPEG-1 stream 3: PES Audio/Video 5: PES Video 7: PES Audio 10: DVD stream

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.30 SetVBIInfo

Synopsis and Command Code

0xC8 (200)
APICommand_Encoder_SetVBIInfo

Description

This command sets various VBI parameters.

Input Parameters

Parameter	Description
P1	VBI Mode: Bit 0 — VBI Mode: 0: Sliced 1: Raw Bits 3–1 — VBI Insertion: 000: Insert in extension_and_user_data. 001: Insert in private packets. 010: Separate stream and user data simultaneously. 111: Separate stream and private data simultaneously. Bits 15–8 — Stream ID. Should be 0xBD (private stream 1).
P2	Number of frames per interrupt (maximum 8). Ignored if in Sliced mode.
P3	Total number of raw VBI frames. Ignored if in Sliced mode. Usually equals 2xP2 for double-buffering.
P4	Start codes. P4 and P5 are interpreted as a code per byte (maximum four codes). The same code can be duplicated if less than four codes are needed.
P5	Stop codes. P4 and P5 are interpreted as a code per byte (maximum four codes). The same code can be duplicated if less than four codes are needed.
P6	Number of lines per frame.
P7	Number of bytes per line.

Return Values

Parameter	Description
R1	Actual frames per interrupt. This is a number between 1 and P2 (see above). Ignored if in Sliced mode.
R2	Actual total number of frames. This is a number between 1 and P3 (see above). Ignored if in Sliced mode.
R3	Offset in memory where the raw VBI data starts. Ignored if in Sliced mode.

P2, P3, R1, R2, and R3 are ignored for the embedded case (extension_and_user_data or private packets).

Usually P3 = 2XP2 for double-buffering.

The following example should be true for the separate case:

```
([P3 x (P6 x P7 + 16)] + 14000) < 176400
[(P3 - 2) x P6 x P7] > 14000
P3 % P2 = 0
P7 % 16 = 0
```

P4 and P5 are interpreted as a code per byte (maximum of 4 codes). The same code can be duplicated if less than 4 codes are needed.

Extra extension_and_user_data will be done in DVD format if the selected stream type using the SetStreamOutputType is DVD Stream.

Return Code

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.2.31 SetVBILineInfo

Synopsis and Command Code

0xB7 (183)
APICommand_Encoder_SetVBILineInfo

Description

This command is used to specify the line number within this field for VBI information, including slicing mode and number of luma and chroma samples in the line.

Input Parameters

Parameter	Description
P1	Line number, as follows: MSB is used to specify whether this is a top field (0) or bottom field (1). Bits 4–0 specify the line number within this field. A value of 0xFFFFFFFF means all lines.
P2	Set VBI Line Information features on/off: 0: Off 1: On
P3	Slicing Mode: 0: No slicing in firmware 1: Closed Caption slicing in firmware
P4	Number of luma samples in this line
P5	Number of chroma samples in this line

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.32 SetVideoBitrate

Synopsis and Command Code

0x95 (149)
APICommand_Encoder_SetVideoBitrate

Description

This command sets video encoding to the specified bit rate. This is the average bit rate at which the video will be output from the encoder. This does not take the audio bit rate into account. The total bit rate = audio + video bit rates. The default is "unknown."

Input Parameters

Parameter	Description
P1	Video bit rate mode: 0: VBR 1: CBR
P2	Video bit rate in bps (e.g., 4000000 = 4 Mbps)
P3	Peak rate in multiples of 400 bps.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.33 SetVideoFrameRate

Synopsis and Command Code

0x8F (143)
APICommand_Encoder_SetVideoFrameRate

Description

This command sets the video frame rate based on the TV encoding format. The actual change takes place after the end of the current GOP.

Input Parameters

Parameter	Description
P1	Frame rate based on TV encoding format: 0: 30 fps (NTSC) 1: 25 fps (PAL)

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.34 SetVideoPID

Synopsis and Command Code

0x8b (139)
APICommand_Encoder_SetVideoPID

Description

This command sets the video PID for Transport streams.

Input Parameters

Parameter	Description
P1	Video PID

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.35 SetVideoResolution

Synopsis and Command Code

0x91 (145)
APICommand_Encoder_SetVideoResolution

Description

This command sets the video encoding resolution based on the width and height values specified by the index in the resolution table described in this document. The default resolution is 720x480.

Input Parameters

Parameter	Description
P1	Height in number of lines: PROP_VIDEO_RESOLUTION_HEIGHT_240 PROP_VIDEO_RESOLUTION_HEIGHT_288 PROP_VIDEO_RESOLUTION_HEIGHT_480 (default) PROP_VIDEO_RESOLUTION_HEIGHT_576
P2	Width in square pixels: PROP_VIDEO_RESOLUTION_WIDTH_720 (default) PROP_VIDEO_RESOLUTION_WIDTH_480 PROP_VIDEO_RESOLUTION_WIDTH_352 PROP_VIDEO_RESOLUTION_WIDTH_320

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.36 StartCapture

Synopsis and Command Code

0x81 (129)
APICommand_Encoder_StartCapture

Description

This command starts the encoding and capture of audio, video, and/or VBI. Encoding parameters must be set prior to issuing this command. This command can capture continuously until a StopCapture is received, or it can be programmed to capture a specific number of frames/samples.

Input Parameters

Parameter	Description
P1	Stream Type: 0 : MPEG_CAPTURE 1: RAW_CAPTURE 2: RAW_PASSTHROUGH 3: VBI_CAPTURE In all cases except MPEG_CAPTURE, you need to set the video resolution and audio encoding parameters (not needed for VBI) before you send the StartCapture command.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.37 StopCapture

Synopsis and Command Code

0x82 (130)
APICommand_Encoder_StopCapture

Description

This command stops the capture and encoding of all incoming video and audio data. Firmware still generates an interrupt (after the command) to copy the end of the stream buffer.

If the Stop Immediately option is not used, stopping the capture could take up to a GOP time (e.g., 0.5 seconds at GOP 15 and NTSC). During this time, StartCapture should not be issued.

Input Parameters

Parameter	Description
P1	Stop capture: 0: Stop at the end of GOP and generate an end-of-stream interrupt. 1: Stop immediately without generating an end-of-stream interrupt.
P2	Stream Type: 0: MPEG_CAPTURE 1: RAW_CAPTURE 2: RAW_PASSTHROUGH 3: VBI_CAPTURE

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.2.38 StopFirmware

Synopsis and Command Code

0xC3 (195)
APICommand_Encoder_StopFirmware

Description

This command stops the firmware. No further API calls can be issued until the firmware is downloaded again.

Input Parameters

None.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3 Decoder Commands

This section details those commands that are specific to a decoding session, including:

Table 5-2: Decoder Commands Summary Table

5.3.1 ExtractVBIData

Synopsis and Command Code

0x19 (025)

ApiCommand_Decoder_ExtractVBIData

Description

This command extracts the VBI data.

Input Parameters

Parameter	Description
P1	Parsing option: Bit 0: Parse from EXTENSION_AND_USER data Bit 1: Parse from private packets

Return Values

Parameter	Description
R1	VBI table location
R2	VBI table size

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.2 GetCurrentFrameInfo

Synopsis and Command Code

0x15 (021)
ApiCommand_Decoder_GetCurrentFrameInfo

Description

This command gets the current timing information from the start of the playback.

Input Parameters

None.

Return Values

Parameter	Description
R1	Current frame count (decode order)
R2	Current video PTS bits 31–0 (display order)
R3	Current video PTS bit 32 (display order)
R4	Current SCR bits 31–0 (display order)
R5	Current SCR bit 32 (display order)

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.3 GetDecoderFWVersion

Synopsis and Command Code

0x11 (017)
ApiCommand_Decoder_GetDecoderFWVersion

Description

This command gets the decoder firmware version information.

Input Parameters

None.

Return Values

Parameter	Description
R1	Firmware version. The structure is 0xXXYYZZZZ, where XX represents the major version number, YY represent the minor version number, and ZZZZ represent the firmware build.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.4 GetDmaFromHostTransferInfo

Synopsis and Command Code

0x09 (009)
`ApiCommand_Decoder_GetDmaFromHostTransferInfo`

Description

This command gets the information for the next data transfer. When information in the buffer drops below 8K in size, the decoder is "starving." This can be used as a means for detecting when the decoder has reached the end of a stream. The DMA types are defined in the DMA_TYPE_ENCODER enum in ivactypes.h.

Input Parameters

None.

Return Values

Parameter	Description
R1	Stream type
R2	Offset address
R3	Maximum amount of transfer
R4	Buffer fullness

Return Code

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.3.5 GetDmaTransferStatus

Synopsis and Command Code

0x0A (010)
ApiCommand_Decoder_GetDmaTransferStatus

Description

This command returns the status of the DMA transfer, and includes information on the type of DMA transfer.

Input Parameters

None.

Return Values

Parameter	Description
R1	DMA Transfer Status: Bit 1 is set to '1' if transfer is completed. Bit 2 is set to '1' if there are any DMA transfer errors. Bit 3 is set to '1' if the link list has any errors.
R2	DMA Type. The DMA types are defined in the DMA_TYPE_ENCODER enum in ivactypes.h: 0 = MPEG 1 = OSD 2 = YUV

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.6 NoOp

Synopsis and Command Code

0x00 (000)
APICommand_Decoder_NoOp

Description

This command performs no operation. Usually it is used to check whether the firmware is alive.

Input Parameters

None.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.7 PausePlayback

Synopsis and Command Code

0x0D (013)
APICommand_Decoder_PausePlayback

Description

This command pauses playback immediately. The decoder will freeze the current frame. The application can use StartPlayback to resume playback.

Once the internal FIFOs are full, no more input data will be accepted. All pending data request interrupts will be masked while paused.

Input Parameters

Parameter	Description
P1	Display Frame: 0: Display last frame. 1: Display a black frame.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.8 SetAudioModeChangeAction

Synopsis and Command Code

0x16 (022)
APICommand_Decoder_SetAudioModeChangeAction

Description

This command sets audio mode, either dual mono or stereo.

Input Parameters

Parameter	Description
P1	Action for dual mono mode.
P2	Action for stereo mode. The actions are defined in ivactypes.h in the FWAPI_PARAM_AudioSenseAction enum, as follows: –1 = No Change 0 = Stereo 1 = Left 2 = Right 3 = Mono 4 = Swap

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.9 SetAudioOutFormat

Synopsis and Command Code

0x1B (027)
APICommand_Decoder_SetAudioOutFormat

Description

This command sets audio output format. This can be done through the GPIO Table, as well (see Section 2.2 of the *iTVC15 Host Driver Development Reference* for more details).

Input Parameters

Parameter	Description
P1	DWORD that represents a bitmask, as follows: Bits 1–0 — Data Channel Width: 0: 16 bits per channel 1: 20 bits per channel 2: 24 bits per channel Bits 7–2 — Reserved Bits 9–8 — Channel Mode: 0: 2 channels 1: 4 channels 2: 6 channels 3: 6 channels with one-line data mode (only for audio_format = 1 and data_width = 1) Bits 11–10 — Reserved Bits 13–12 — Channel Audio Format: 0: Right-justified MSB-first mode 1: Left-justified MSF-first mode 2: I^2S mode Bits 15–14 — Reserved Bits 21–16 — Justify Count (in case of right-justified audio format) Bits 31–22 — Reserved

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.10 SetAVsyncOffset

Synopsis and Command Code

0x1C (028)
APICommand_Decoder_SetAVsyncOffset

Description

This command sets the number of 90-KHz ticks to delay audio or video.

Input Parameters

Parameter	Description
P1	Stepping Unit: 0: Normal, sync audio and video exactly >0: Delay video <0: Delay audio (i.e., advance video)

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.11 SetDecodeMode

Synopsis and Command Code

0x1A (026)
APICommand_Decoder_SetDecodeMode

Description

This command sets the decoder source to compressed data from the host, raw data passed from the encoder (pass-through), or raw data passed from the host (display mode).

Note that in Encode Pass-through mode, the host needs to make sure that encode resolution and sampling frequency is the same as what is passed to this command.

Input Parameters

Parameter	Description
P1	Decoder Mode: 0: Decoding compressed data 1: Encoder pass-through 2: Display mode
P2	Width in pixels of video source (required if P1 is different than 0)
P3	Height in lines of the video source (required if P1 is different than 0)
P4	Bitmask specifying the audio parameters based on the union FWAPI_PARAM_MPEGAudioEncodingParameters in ivactypes.h. The following fields are reserved: dwLayer dwBitrate joint stereo mode dwModeExtension dwEmphasis dwErrProtection dwCopyright dwOriginal

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.12 SetDisplayMode

Synopsis and Command Code

0x10 (016)
APICommand_Decoder_SetDisplayMode

Description

This command sets the output display format (NTSC or PAL).

Input Parameters

Parameter	Description
P1	Display format: 0: NTSC 1: PAL

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.13 SetDMABlockSize

Synopsis and Command Code

0x08 (008)
ApiCommand_Decoder_SetDMABlockSize

Description

This command sets the transfer size for every DMA access. This size can be overridden when actually issuing the DMA transfer command.

Input Parameters

Parameter	Description
P1	Block size in bytes for every DMA transfer.

Return Values

None.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.14 SetDMAFromHostLinkAddress

Synopsis and Command Code

0x0B (011)
APICommand_Decoder_SetDMAFromHostLinkAddress

Description

Firmware initiates the DMA transfer.

Input Parameters

Parameter	Description
P1	Physical address of the link list.
P2	Size of data in the list.
P3	DMA Type. The DMA types are defined in the DMA_TYPE_ENCODER enum in ivactypes.h, as follows: 0 = MPEG 1 = OSD 2 = YUV

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.15 SetEventNotification

Synopsis and Command Code

0x17 (023)
APICommand_Decoder_SetEventNotification

Description

This command sets the event notification function.

Input Parameters

Parameter	Description
P1	Event type. One of the values in the enum FWAPI_PARAM_DecoderEventType, as follows: 0 = Audio Mode Change. These event notifies the host that the audio mode has changed between stereo and dual channel, or vice versa.
P2	Event Notification: 0: Off 1: On
P3	Interrupt bit assigned to the event. The host needs to unmask this bit. Please refer to the "Data Transfer" section of the *iTVC15 Host Driver Development Reference* for more information.
P4	Mailbox slot assigned to the event. A value of –1 means that this event does not require a mailbox slot.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.16 SetNumberOfDisplayBuffers

Synopsis and Command Code

0x18 (024)
APICommand_Decoder_SetNumberOfDisplayBuffers

Description

This command sets the number of display buffers used by the decoder — either six or nine. If nine buffers are used, the decoder allows for decoding all frames in reverse playback, even when the GOP size is 15, at the expense of decreasing the usability of the OSD.

Input Parameters

Parameter	Description
P1	Number of Display Buffers: 0: 6 buffers 1: 9 buffers

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.17 SetStreamInputPort

Synopsis and Command Code

0x14 (020)

APICommand_Decoder_SetStreamInputPort

Description

This command sets the decoded data stream input port.

Input Parameters

Parameter	Description
P1	Input Port: 0: Memory (default) 1: Streaming Port

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.18 SetTrickMode

Synopsis and Command Code

0x03 (003)

APICommand_Decoder_SetTrickMode

Description

This command Initiates a trick mode operation. Two separate modes are possible, Smooth and Coarse, as follows:

- **Smooth mode.** The speed is other than normal. The host should transfer the complete stream and the FW will drop the pictures that are not required.
- **Coarse mode.** The host should drop frames based on indexing to achieve a certain speed (see *iTVC15 Host Driver Development Reference* for more information).

Note: P4 is required for reverse play only. It specifies the total number of B-frames in each GOP (refer to frame ordering in reverse playback in *iTVC15 Host Driver Development Reference* for more details).

Input Parameters

Parameter	Description
P1	Speed: Bit 31 — Speed: 0: Slow 1: Fast Bits 7–0 — Speed factor: 0: Normal 1: Used for fast only. Interpreted as 1.5X y: For fast yX, for slow 1/yX
P2	Playback Direction: 0: Forward 1: Backward
P3	Picture Mask. The allowed values are: 1: I-frames only 3: I- and P-frames 7: All frames
P4	Number of B-frames per GOP
P5	Audio Mute: 0: Off 1: On
P6	Display Mode: 0: Frame 1: Field

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds;

API_RC_ERROR_UNKNOWN otherwise

5.3.19 StartPlayback

Synopsis and Command Code

0x01 (001)
APICommand_Decoder_StartPlayback

Description

Play begins or resumes the Parser to continue sending data to the decoder buffers. If resuming from a stopped state, data sent to the decoders may be discontinuous, in which case each decoder is responsible for finding the first complete access unit in the bit stream.

Input Parameters

Parameter	Description
P1	Frame # in the first GOP, starting from 0. If a non-zero value is set, the display skips the first P1 frames.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.20 Step

Synopsis and Command Code

0x05 (005)
ApiCommand_Decoder_Step

Description

Each time this command is called, the playback advances by one unit (field or frame) in the current playback direction.

Input Parameters

Parameter	Description
P1	Stepping Unit: 0: Field 1: Frame

Return Values

None.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.21 StopDecFirmware

Synopsis and Command Code

0x0E (014)
ApiCommand_Decoder_StopDecFirmware

Description

This command stops the firmware. No further API calls can be issued until the decoder firmware is re-downloaded.

Input Parameters

None.

Return Values

None.

Return Codes

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.3.22 StopPlayback

Synopsis and Command Code

0x02 (002)
APICommand_Decoder_StopPlayback

Description

This command pauses the Parser and further terminates all commands in execution and clears all decoder buffers. If P2 and P3 are set, the firmware will stop when it reaches the specified PTS.

Input Parameters

Parameter	Description
P1	Display frame: 0: Display last frame 1: Display black frame
P2	PTS low
P3	PTS high (just the LSB matters)

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4 Display Commands

This section details those commands that are specific to controlling the display, including:

Table 5-3: Display Commands Summary Table

5.4.1 BLTCopy

Synopsis and Command Code

```
0x52 (082)
APICommand_Display_BLTCopy
```

Description

This command is used to perform a BLT Copy operation.

Input Parameters

Parameter	Description
P1	Pixel ROP code: 0000: 0 0001: !DST & !SRC 0010: !DST & SRC 0011: !DST 0100: DST & !SRC 0101: !SRC 0110: DST xor SRC 0111: !DST + !SRC 1000: !DST & !SRC 1001: DST xnor SRC 1010: SRC 1011: !DST + SRC 1100: DST 1101: DST + !SRC 1110: DST + SRC 1111: 1
P2	Alpha blending determines the value of A_{final} : 01: New Alpha is Source Alpha ($A_{final} = A_{src}$) 10: New Alpha is Destination Alpha ($A_{final} = A_{dst}$) 11: New Alpha is Blended Source and Destination Alpha if (A_{src} == A_{dst}== 0h) A_{final} = 0, else $A_{final} = A_{src}$ * Adst + 1 where A_{src} = Source Pixel Alpha A_{dst} = Destination Pixel Alpha

Parameter	Description
P3	Pixel blending: Pixels are blended thus: $P_{final} = (F_{src}*P_{src} + F_{dst}* P_{dst}) >> 8$ where P_{final} = Pixel Output P_{src} = Source Pixel P_{dst} = Destination Pixel F_{src} = Source Blend Factor F_{dst} = Destination Blend Factor and the blend factors (F_{src} and F_{dst}) are based on the source (A_{src}) and destination (A_{dst}) pixel alpha values, for each value of pixel blending, as follows: 00 F_{src} = 100h F_{dst} = 0h 01 A_{src} = 0h: F_{src}= 0h; F_{dst}= 100h A_{src} = 0FFh: F_{src}= 100h; F_{dst}= 0h 0h < A_{src} < 0FFh: F_{src}= A_{src} + 1; F_{dst}= 100h – A_{src} – 1 10 A_{dst} = 0h: F_{src}= 100h; F_{dst}= 0h A_{dst} = 0FFh: F_{src}= 0h; F_{dst}= 100h 0h < A_{dst} < 0FFh: F_{src}= 100h – A_{dst} – 1; F_{dst}= A_{dst} + 1 11 A_{src} = 0h: F_{src}= 0h A_{src} = 0FFh: F_{src}= 100h 0h < A_{src} < 0FFh: F_{src}= A_{src} + 1 A_{dst} = 0h: F_{dst}= 0h A_{dst} = 0FFh: F_{dst}= 100h 0h < A_{dst} < 0FFh: F_{dst}= A_{dst} + 1
P4	Width in pixels.
P5	Height in lines.
P6	The 32-bit value for the destination pixel mask.
P7	Starting address of the destination rectangle.
P8	Destination stride in dwords.
P9	Source stride in dwords.
P10	Starting address of the source rectangle.

Return Values

None.

Return Code

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.4.2 BLTExpandText

Synopsis and Command Code

```
0x54 (084)
APICommand_Display_BLTExpandText
```

Description

This command is used to perform a BLT operation where the source is 8-bit alpha text.

Input Parameters

Parameter	Description
P1	Pixel ROP code: 0000: 0 0001: !DST & !SRC 0010: !DST & SRC 0011: !DST 0100: DST & !SRC 0101: !SRC 0110: DST xor SRC 0111: !DST + !SRC 1000: !DST & !SRC 1001: DST xnor SRC 1010: SRC 1011: !DST + SRC 1100: DST 1101: DST + !SRC 1110: DST + SRC 1111: 1
P2	Alpha blending determines the value of A_{final} : 01: New Alpha is Source Alpha ($A_{final} = A_{src}$) 10: New Alpha is Destination Alpha ($A_{final} = A_{dst}$) 11: New Alpha is Blended Source and Destination Alpha if ($A_{src} == A_{dst} == 0h$) $A_{final} = 0$, else $A_{final} = A_{src} * Adst + 1$ where A_{src} = Source Pixel Alpha A_{dst} = Destination Pixel Alpha

Parameter	Description
P3	Pixel blending: Pixels are blended thus: $P_{final} = (F_{src}*P_{src} + F_{dst}* P_{dst}) >> 8$ where P_{final} = Pixel Output P_{src} = Source Pixel P_{dst} = Destination Pixel F_{src} = Source Blend Factor F_{dst} = Destination Blend Factor and the blend factors (F_{src} and F_{dst}) are based on the source (A_{src}) and destination (A_{dst}) pixel alpha values, for each value of pixel blending, as follows: 00 F_{src} = 100h F_{dst} = 0h 01 A_{src} = 0h: F_{src}= 0h; F_{dst}= 100h A_{src} = 0FFh: F_{src}= 100h; F_{dst}= 0h 0h < A_{src} < 0FFh: F_{src}= A_{src} + 1; F_{dst}= 100h – A_{src} – 1 10 A_{dst} = 0h: F_{src}= 100h; F_{dst}= 0h A_{dst} = 0FFh: F_{src}= 0h; F_{dst}= 100h 0h < A_{dst} < 0FFh: F_{src}= 100h – A_{dst} – 1; F_{dst}= A_{dst} + 1 11 A_{src} = 0h: F_{src}= 0h A_{src} = 0FFh: F_{src}= 100h 0h < A_{src} < 0FFh: F_{src}= A_{src} + 1 A_{dst} = 0h: F_{dst}= 0h A_{dst} = 0FFh: F_{dst}= 100h 0h < A_{dst} < 0FFh: F_{dst}= A_{dst} + 1
P4	Width in pixels.
P5	Height in lines.
P6	The 32-bit value for the destination pixel mask.
P7	Starting address of the destination rectangle.
P8	Destination stride in dwords.
P9	Source stride in dwords.
P10	Starting address of the source rectangle.
P11	32-bot color value for the fill operation.

Return Values

None.

Return Code

`API_RC_SUCCESS` if the function succeeds.
`API_RC_ERROR_UNKNOWN` otherwise.

5.4.3 BLTFillColor

Synopsis and Command Code

0x53 (083)
APICommand_Display_BLTFillColor

Description

This command is used to perform a BLT Fill Color operation.

Input Parameters

Parameter	Description
P1	Pixel ROP code: 0000: 0 0001: !DST & !SRC 0010: !DST & SRC 0011: !DST 0100: DST & !SRC 0101: !SRC 0110: DST xor SRC 0111: !DST + !SRC 1000: !DST & !SRC 1001: DST xnor SRC 1010: SRC 1011: !DST + SRC 1100: DST 1101: DST + !SRC 1110: DST + SRC 1111: 1
P2	Alpha blending determines the value of A_{final} : 01: New Alpha is Source Alpha ($A_{final} = A_{src}$) 10: New Alpha is Destination Alpha ($A_{final} = A_{dst}$) 11: New Alpha is Blended Source and Destination Alpha if ($A_{src} == A_{dst} == 0h$) $A_{final} = 0$, else $A_{final} = A_{src} * Adst + 1$ where A_{src} = Source Pixel Alpha A_{dst} = Destination Pixel Alpha

Parameter	Description
P3	Pixel blending: Pixels are blended thus: $P_{final} = (F_{src}*P_{src} + F_{dst}* P_{dst}) >> 8$ where P_{final} = Pixel Output P_{src} = Source Pixel P_{dst} = Destination Pixel F_{src} = Source Blend Factor F_{dst} = Destination Blend Factor and the blend factors (F_{src} and F_{dst}) are based on the source (A_{src}) and destination (A_{dst}) pixel alpha values, for each value of pixel blending, as follows: 00 F_{src} = 100h F_{dst} = 0h 01 A_{src} = 0h: F_{src}= 0h; F_{dst}= 100h A_{src} = 0FFh: F_{src}= 100h; F_{dst}= 0h 0h < A_{src} < 0FFh: F_{src}= A_{src} + 1; F_{dst}= 100h – A_{src} – 1 10 A_{dst} = 0h: F_{src}= 100h; F_{dst}= 0h A_{dst} = 0FFh: F_{src}= 0h; F_{dst}= 100h 0h < A_{dst} < 0FFh: F_{src}= 100h – A_{dst} – 1; F_{dst}= A_{dst} + 1 11 A_{src} = 0h: F_{src}= 0h A_{src} = 0FFh: F_{src}= 100h 0h < A_{src} < 0FFh: F_{src}= A_{src} + 1 A_{dst} = 0h: F_{dst}= 0h A_{dst} = 0FFh: F_{dst}= 100h 0h < A_{dst} < 0FFh: F_{dst}= A_{dst} + 1
P4	Width in pixels.
P5	Height in lines.
P6	The 32-bit value for the destination pixel mask.
P7	Starting address of the destination rectangle.
P8	Destination stride in dwords.
P9	32-bit color value for the fill operation

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.4 GetGlobalAlpha

Synopsis and Command Code

0x4A (074)
APICommand_Display_GetGlobalAlpha

Description

This command returns the current OSD's 8-bit Global Alpha.

Input Parameters

None.

Return Values

Parameter	Description
R1	Global Alpha: 0: Disable 1: Enable
R2	8-bit value for Global Alpha.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.5 GetGraphicsBufferSpace

Synopsis and Command Code

0x41 (065)
APICommand_Display_GetGraphicsBufferSpace

Description

This command is used to determine the graphics buffer base address and size. This memory is contiguous and is to be managed by the driver/application for OSD buffers, scratch buffers for OSD elements, and any BLT command lists.

Input Parameters

None.

Return Values

Parameter	Description
R1	The base address of the OSD memory.
R2	The maximum size of the OSD memory available.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.6 GetOSDAlphaContent

Synopsis and Command Code

0x61 (097)
APICommand_Display_GetOSDAlphaContent

Description

This command gets the Alpha content in the iven Alpha Content Index.

Input Parameters

None.

Return Values

Parameter	Description
R1	OSD Alpha Content Index (between 0 and 15)

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.7 GetOSDDeflickerState

Synopsis and Command Code

0x4F (079)

APICommand_Display_GetDeflickerState

Description

This command gets the current deflicker state.

Input Parameters

None.

Return Values

Parameter	Description
R1	FRM State: 0: Disabled 1: Enabled

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.8 GetOSDDisplayedBuffer

Synopsis and Command Code

0x46 (070)
APICommand_Display_GetOSDDisplayedBuffer

Description

This command gets the coordinates of the current area in the OSD being blended with the video.

Input Parameters

None.

Return Values

Parameter	Description
R1	Start address of the OSD buffer.
R2	Stride of the OSD buffer in pixels.
R3	Number of lines in the OSD buffer.
R4	Initial X offset within the OSD buffer.
R5	Initial Y offset within the OSD buffer.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.9 GetOSDPixelFormat

Synopsis and Command Code

0x42 (066)
APICommand_Display_GetOSDPixelFormat

Description

This command gets the Pixel Format being used for the OSD buffer(s).

Input Parameters

None.

Return Values

Parameter	Description
R1	Pixel Format. Allowed values are: 0: 8-bit index 4: ARGB 8:8:8:8

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.10 GetOSDScreenCoordinates

Synopsis and Command Code

0x48 (072)
APICommand_Display_GetOSDScreenCoordinates

Description

This command gets the coordinates of the current screen area being blended with the video.

Input Parameters

None.

Return Values

Parameter	Description
R1	X-coordinate of the top left pixel.
R2	Y-coordinate of the top left pixel.
R3	X-coordinate of the bottom right pixel.
R4	Y-coordinate of the bottom right pixel.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.11 GetOSDState

Synopsis and Command Code

0x44 (068)
APICommand_Dipslay_GetOSDState

Description

This command is used to retrieve the current state of the OSD.

Input Parameters

None.

Return Values

Parameter	Description
R1	OSD State: Bit 0 — On/Off 0: Disable 1: Enable Bits 2–1 — OSD Alpha Control Bits 5–3 — OSD Pixel Format

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.12 MoveOSDBuffer

Synopsis and Command Code

0x4C (078)
APICommand_Display_MoveOSDBuffer

Description

This command moves the coordinates of the starting position of the blending area inside the displayed buffer.

Input Parameters

Parameter	Description
P1	X offset within the OSD buffer.
P2	Y offset within the OSD buffer.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.13 SetGlobalAlpha

Synopsis and Command Code

0x4B (075)
APICommand_Display_SetGlobalAlpha

Description

This command updates the OSD's 8-bit Global Alpha.

Input Parameters

Parameter	Description
P1	Global Alpha: 0: Disable 1: Enable
P2	8-bit value for Global Alpha.
P3	Local Alpha: 0: Enable 1: Disable

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.14 SetOSDAlphaContent

Synopsis and Command Code

0x62 (098)
APICommand_Display_SetOSDAlphaContent

Description

This command sets the Alpha content in the given OSD Alpha Content Index.

Input Parameters

Parameter	Description
P1	OSD Alpha Content Index (between 0 and 15)

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.15 SetOSDChromaKey

Synopsis and Command Code

0x60 (096)
APICommand_Display_SetOSDChromaKey

Description

This command turns Chroma Key ON or OFF, and sets the Chroma Key color.

Input Parameters

Parameter	Description
P1	Chroma Key: 0: Off 1: On
P2	Chroma Key Color

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.16 SetOSDDeflickerState

Synopsis and Command Code

0x50 (080)
APICommand_Display_SetDeflickerState

Description

This command is used to enable/disable the deflicker filter.

Input Parameters

Parameter	Description
P1	Deflicker Filter: 0: Disable 1: Enable

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.17 SetOSDDisplayedBuffer

Synopsis and Command Code

0x47 (071)
APICommand_Display_SetOSDDisplayedBuffer

Description

This command sets/updates the coordinates of the current area in the OSD being blended with the video.

Input Parameters

Parameter	Description
P1	Start address of the OSD buffer.
P2	Stride of the OSD buffer in pixels.
P3	Number of lines in the OSD buffer.
P4	Initial X offset within the OSD buffer.
P5	Initial Y offset within the OSD buffer.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.18 SetOSDPixelFormat

Synopsis and Command Code

0x43 (067)
APICommand_Display_SetOSDPixelFormat

Description

This command sets/changes the Pixel Format for the OSD buffer(s).

Input Parameters

Parameter	Description
P1	Pixel format: 0: 8-bit index 4: ARGB 8:8:8:8

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.19 SetOSDScreenCoordinates

Synopsis and Command Code

0x49 (073)
APICommand_Display_SetOSDScreenCoordinates

Description

This command sets/updates the coordinates of the screen area being blended with the video.

Input Parameters

Parameter	Description
P1	X-coordinate of the top left pixel.
P2	Y-coordinate of the top left pixel.
P3	X-coordinate of the bottom right pixel.
P4	Y-coordinate of the bottom right pixel.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.20 SetOSDState

Synopsis and Command Code

0x45 (069)
APICommand_Display_SetOSDState

Description

This command is used to enable/disable OSD.

Input Parameters

Parameter	Description
P1	OSD: 0: Disable 1: Enable

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.

5.4.21 SetVideoScreenCoordinates

Synopsis and Command Code

0x56 (086)
APICommand_Display_SetVideoScreenCoordinates

Description

This command sets the main picture window for video output on the screen. The default is full screen display. For reduced video window sizes, the window can be positioned anywhere on the screen, but it is required that the whole window be visible — i.e., the window can't be "pushed" off the edge of the screen.

Input Parameters

Parameter	Description
P1	Specifies the width of the video window.
P2	Specifies the height of the video window.
P3	Specifies the horizontal position of the top left corner of the video window.
P4	Specifies the vertical position of the top left corner of the video window.

Return Values

None.

Return Code

API_RC_SUCCESS if the function succeeds.
API_RC_ERROR_UNKNOWN otherwise.